

Web Technology

Web technology is the development and use of various tools and techniques that enable communication and interaction between different types of devices over a network, such as the internet. Web technology includes:

- **Markup languages:** These are the languages that define the structure, content, and presentation of web pages. Examples are HTML, XML, CSS, and JavaScript.
- **Programming interfaces:** These are the interfaces that allow web developers to create dynamic and interactive web applications. Examples are CGI, ASP, PHP, JSP, and AJAX.
- **Standards:** These are the rules and guidelines that ensure the compatibility and interoperability of web technologies. Examples are HTTP, HTTPS, TCP/IP, URL, URI, and DNS.
- **Web browsers:** These are the programs that display text, data, images, animation, and video on the internet. Examples are Chrome, Firefox, Safari, and Edge.
- **Web servers:** These are the programs that store and deliver web pages and other resources to web browsers. Examples are Apache, IIS, and Nginx.
- **Web services:** These are the services that provide data and functionality to web applications and other web services. Examples are REST, SOAP, and JSON.

Unit 1 - Introduction to Web Technology

- Web technology is the use of various tools and protocols to create and deliver information and services over the internet.
- Web technology consists of two main components: the client and the server.
- The client is the device or software that requests and displays web content, such as a web browser or a mobile app.
- The server is the device or software that responds to client requests and provides web content, such as a web server or a web application.
- Web content is the information and data that is transmitted between the client and the server, such as text, images, audio, video, etc.
- Web content is formatted using various languages and standards, such as HTML, CSS, JavaScript, XML, JSON, etc.
- Web technology enables various applications and services, such as e-commerce, social media, online education, gaming, etc.
- Web technology is constantly evolving and improving, with new tools and protocols being developed and adopted.

Introduction to Web Technology

- Web technology is the development of the mechanism that allows two or more computer devices to communicate over a network.
- Web technology includes markup languages, programming interfaces, standards to define document identity and display.
- Web technology refers to the various tools and techniques that are utilized in the process of communication between different types of devices over the internet.
- A web browser is used to access web pages. Web browsers can be defined as programs that display text, data, pictures, animation, and video on the internet.
- Web technology is a method of communication unique to computers through the use of binary codes and directions.
- Web technology is constantly evolving and improving, with new trends and features emerging every year. Some of the current trends in web technology are:
 - Responsive web design: This is the practice of designing web pages that can adapt to different screen sizes and devices, such as desktops, laptops, tablets, and smartphones.
 - Progressive web apps: These are web applications that can work offline, load faster, and provide a native app-like experience to the users.
 - WebAssembly: This is a new format for executable code on the web, which allows for faster and more efficient performance of complex tasks, such as games, graphics, and audio processing.
 - Artificial intelligence: This is the use of machine learning and natural language processing to create web applications that can understand and interact with the users, such as chatbots, voice assistants, and recommendation systems.

Web Development Strategies

Web development is the process of creating, designing, and maintaining websites and web applications. Web development strategies are the methods and techniques that web developers use to achieve their goals and deliver high-quality products to their clients and users. Some of the web development strategies are:

- **Planning and analysis:** This is the first and most important step in any web development project. It involves defining the scope, objectives, requirements, and specifications of the project, as well as identifying the target audience, competitors, and market trends. Planning and analysis also include choosing the appropriate tools, technologies, frameworks, and platforms for the project, as well as creating a realistic timeline and budget.
- **Design and prototyping:** This is the stage where the web developer creates the visual and interactive elements of the website or web application,

such as the layout, color scheme, typography, images, icons, animations, and navigation. Design and prototyping also involve testing the usability, accessibility, and responsiveness of the website or web application, as well as getting feedback from the client and users.

- **Development and testing:** This is the stage where the web developer writes the code and implements the functionality of the website or web application, using the chosen tools, technologies, frameworks, and platforms. Development and testing also involve debugging, optimizing, and validating the code, as well as ensuring the compatibility, security, and performance of the website or web application across different browsers, devices, and platforms.
- **Deployment and maintenance:** This is the stage where the web developer launches the website or web application to the public, using a web server, a hosting service, or a cloud platform. Deployment and maintenance also involve monitoring, updating, and improving the website or web application, as well as fixing any issues, errors, or bugs that may arise.

History of Web

- The World Wide Web (WWW) is a system of interconnected documents that can be accessed through the Internet using a web browser.
- The Web was invented by Tim Berners-Lee, a British scientist, in 1989, while working at CERN, a European research organization .
- The Web was originally conceived and developed to meet the demand for automated information-sharing between scientists in universities and institutes around the world.
- The Web is based on several concepts and technologies, the most fundamental of which are:
 - HyperText: a way of linking and accessing information of various kinds as a web of nodes in which the user can browse at will.
 - HyperText Markup Language (HTML): a language for creating and formatting web pages.
 - HyperText Transfer Protocol (HTTP): a protocol for transferring web pages and other data over the Internet.
 - Uniform Resource Locator (URL): a unique address for identifying and locating a web resource.
- The first web browser, called WorldWideWeb, was also created by Berners-Lee in 1990. It was later renamed Nexus to avoid confusion with the Web itself.
- The first web server, which hosted the first web page, was installed at CERN in 1990. The first web page, which described the project and its goals, can still be viewed at <http://info.cern.ch/hypertext/WWW/TheProject.html>.
- The Web became publicly available in 1991, when CERN announced that the Web would be free and open for anyone to use.
- The Web grew rapidly in popularity and usage in the 1990s, with the emergence of more web browsers, such as Mosaic, Netscape, and Internet

Explorer, and more web sites, such as Yahoo!, Amazon, and eBay.

- The Web also enabled new forms of communication, collaboration, and creativity, such as email, social media, blogs, wikis, podcasts, and web applications.
- The Web continues to evolve and expand, with new standards, technologies, and trends, such as HTML5, CSS3, JavaScript, AJAX, Web 2.0, Web 3.0, Semantic Web, and Web of Things.

History of Internet

- The internet is a system of interconnected computer networks that allows communication and data exchange across the world.
- The origins of the internet can be traced back to the 1950s, when the United States was in the Cold War with the Soviet Union and wanted to create a reliable and decentralized communication system that could survive a nuclear attack .
- The first precursor of the internet was the ARPANET, which stands for Advanced Research Projects Agency Network. It was a project funded by the US Department of Defense that connected four universities in 1969 .
- The ARPANET used a protocol called NCP (Network Control Protocol) to transfer data between the computers. However, NCP had some limitations and could not support different types of networks.
- In the 1970s, two researchers, Bob Kahn and Vint Cerf, developed a new protocol called TCP/IP (Transmission Control Protocol/Internet Protocol) that could interconnect different networks and create a network of networks, or an internet .
- TCP/IP was adopted by the ARPANET in 1983 and became the standard for data transmission over the internet. TCP/IP is still used today as the basis of the internet.
- The internet grew rapidly in the 1980s and 1990s, as more networks joined the system and more applications were developed, such as email, file transfer, and the World Wide Web .
- The World Wide Web, or the web, was invented by Tim Berners-Lee in 1989. It is a system of hypertext documents that can be accessed through a web browser and linked by URLs (Uniform Resource Locators) .
- The web made the internet more accessible and popular to the general public, as it allowed users to access information and services from different websites using a graphical interface .
- The internet has revolutionized many aspects of human society, such as communication, education, commerce, entertainment, and social interaction. It has also raised some challenges and issues, such as security, privacy, censorship, and digital divide.

Protocols Governing Web

- Protocols are a set of rules or standards that enable different devices or applications to communicate over a network, such as the internet.
- The web is a system of interlinked documents or resources that are accessed through the internet using web browsers and web servers.
- The web relies on various protocols to function properly and efficiently. Some of the most common protocols governing the web are:
 - **TCP/IP (Transmission Control Protocol/Internet Protocol)**: This is the fundamental protocol suite that provides the basic functionality of the internet. It consists of two main protocols: TCP and IP. TCP is responsible for ensuring reliable and ordered delivery of data packets between devices, while IP is responsible for routing and addressing the packets across the network. TCP/IP also supports other protocols that provide additional services, such as DNS, HTTP, FTP, etc.
 - **DNS (Domain Name System)**: This is a protocol that translates human-readable domain names, such as `www.example.com`, into numerical IP addresses, such as `192.168.1.1`, that are used by devices to locate and access resources on the internet. DNS also provides other information, such as the mail server or the name server of a domain. DNS operates in a hierarchical and distributed manner, using a network of servers that store and update the mappings between domain names and IP addresses.
 - **HTTP (HyperText Transfer Protocol)**: This is a protocol that defines how web browsers and web servers communicate and exchange information, such as web pages, images, videos, etc. HTTP is based on a request-response model, where a web browser sends a request for a resource to a web server, and the web server responds with the resource or an error message. HTTP also supports various methods, such as GET, POST, PUT, DELETE, etc., that indicate the action to be performed on the resource. HTTP also supports various headers, such as Content-Type, Content-Length, Cookie, etc., that provide additional information about the request or the response.
 - **FTP (File Transfer Protocol)**: This is a protocol that enables the transfer of files between devices over a network, such as the internet. FTP uses two connections: a control connection and a data connection. The control connection is used to establish the session, authenticate the user, and send commands, while the data connection is used to transfer the files. FTP supports various modes, such as binary, ASCII, passive, active, etc., that affect how the data is transferred and the ports used.

Writing Web Projects

A web project is a collection of files and folders that are used to create a website or a web application. A web project typically consists of the following components:

- **HTML files:** These are the files that contain the structure and content of the web pages. HTML stands for HyperText Markup Language and it is the standard language for creating web pages.
- **CSS files:** These are the files that contain the style and layout of the web pages. CSS stands for Cascading Style Sheets and it is the language that defines how HTML elements are displayed on the screen.
- **JavaScript files:** These are the files that contain the logic and interactivity of the web pages. JavaScript is a scripting language that can manipulate the HTML and CSS elements and respond to user events.
- **Images, fonts, icons, and other media files:** These are the files that provide the visual elements of the web pages, such as pictures, logos, icons, etc.
- **Frameworks and libraries:** These are the files that provide additional functionality and features to the web project, such as Bootstrap, jQuery, React, etc. Frameworks and libraries are usually written in HTML, CSS, or JavaScript and they can be downloaded from the internet or installed using a package manager.
- **Configuration files:** These are the files that specify the settings and options of the web project, such as the name, version, dependencies, scripts, etc. Configuration files are usually written in JSON, XML, or YAML formats and they can be read by the web server, the browser, or the development tools.

To write a web project, one needs to follow these steps:

- **Plan the web project:** This involves defining the purpose, scope, audience, and features of the web project. It also involves creating a wireframe or a mockup of the web pages, which is a sketch or a diagram that shows the layout and elements of the web pages.
- **Choose the tools and technologies:** This involves selecting the tools and technologies that will be used to create the web project, such as the text editor, the web server, the browser, the frameworks, and the libraries. It also involves setting up the development environment, which is the system and the software that are used to write, test, and debug the web project.
- **Write the code:** This involves writing the HTML, CSS, and JavaScript code that will create the web pages and the web application. It also involves testing and debugging the code, which is the process of finding and fixing errors and bugs in the code.
- **Deploy the web project:** This involves uploading the web project files to a web server, which is a computer that hosts the web project and makes it accessible to the internet. It also involves registering a domain name, which is the address of the web project on the internet, such as

www.example.com.

Connecting to Internet

- The Internet is a global network of computers and devices that can communicate and share information with each other.
- To connect to the Internet, a device needs to have a network interface card (NIC) that can send and receive data packets over a physical or wireless medium.
- A device also needs to have an Internet Protocol (IP) address that identifies it uniquely on the network and a Domain Name System (DNS) server that can translate domain names into IP addresses.
- A device can connect to the Internet through different types of service providers, such as Internet Service Providers (ISPs), Mobile Network Operators (MNOs), or Wireless Fidelity (Wi-Fi) hotspots.
- A device can use different protocols and technologies to access the Internet, such as Transmission Control Protocol/Internet Protocol (TCP/IP), Hypertext Transfer Protocol (HTTP), Secure Sockets Layer (SSL), or Virtual Private Network (VPN).
- A device can use different applications and browsers to access the Internet, such as web browsers, email clients, social media platforms, or online games.

Introduction to Internet Services

Internet services are the applications or functions that allow users to access, exchange, or create information and media over the internet. The internet is a global system of interconnected computer networks that use the Internet Protocol (IP) and the Transmission Control Protocol (TCP) to communicate and transfer data. Some of the main categories of internet services are:

- **Communication services:** These services enable users to communicate with other individuals or groups using various methods, such as email, instant messaging, voice over IP, video conferencing, social networking, etc. Communication services are widely used for personal, professional, and educational purposes. Examples of communication services are Gmail, Skype, WhatsApp, Facebook, etc.
- **Information services:** These services provide users with access to a large amount of information and knowledge on various topics, such as news, education, entertainment, sports, health, etc. Information services are useful for learning, research, and awareness. Examples of information services are Wikipedia, Google, YouTube, Khan Academy, etc.
- **Web services:** These services allow users to interact with web pages or applications that are hosted on web servers and accessed through web browsers. Web services can offer various functionalities, such as online shopping, banking, gaming, booking, etc. Web services are convenient and

efficient for users and businesses. Examples of web services are Amazon, PayPal, Netflix, Airbnb, etc.

- **Cloud services:** These services allow users to store, access, or process data and resources on remote servers that are connected to the internet. Cloud services can offer various benefits, such as scalability, reliability, security, and cost-effectiveness. Cloud services can be classified into different types, such as software as a service (SaaS), platform as a service (PaaS), and infrastructure as a service (IaaS). Examples of cloud services are Google Drive, Dropbox, Microsoft Azure, Amazon Web Services, etc.

Introduction to Internet tools

- Internet tools are electronic instruments based on Internet technology with which parties can interact.
- Internet tools can be classified into different categories, such as:
 - Web tools: These are tools that allow users to access, create, and share information on the World Wide Web, such as browsers, search engines, web pages, blogs, wikis, podcasts, etc.
 - Communication tools: These are tools that enable users to communicate with each other over the Internet, such as email, instant messaging, chat, videoconference, social media, etc.
 - Collaboration tools: These are tools that facilitate users to work together on a common project or task over the Internet, such as cloud computing, file sharing, online document editing, project management, etc.
 - Education tools: These are tools that support users to learn and teach over the Internet, such as online courses, learning management systems, educational games, simulations, etc.
 - Entertainment tools: These are tools that provide users with fun and leisure activities over the Internet, such as online games, music, videos, streaming services, etc.
- Internet tools have various benefits and challenges for users, such as:
 - Benefits: Internet tools can enhance the accessibility, availability, diversity, and quality of information and communication. They can also improve the efficiency, productivity, creativity, and innovation of users. They can also offer new opportunities for learning, teaching, working, and entertainment.
 - Challenges: Internet tools can also pose some risks and drawbacks for users, such as privacy, security, reliability, and ethical issues. They can also create digital divide, information overload, and cyber addiction. They can also require technical skills, infrastructure, and resources to use them effectively.

Introduction to client-server computing

- Client-server computing is a software architecture model that divides an application into two parts: client systems and server systems .
- A client system is a program or device that requests a service or resource from a server system, which is a program or device that provides the service or resource .
- Client and server systems can communicate over a computer network or on the same computer .
- Client-server computing allows for the distribution of tasks or workloads among different computers, which can improve performance, scalability, security, and reliability .
- Examples of client-server applications are web browsers and web servers, email clients and email servers, online games, and database systems.

Core Java

Core Java is the term used to refer to the basic concepts and features of the Java programming language. It is the part of Java SE (Standard Edition) platform, which is used for developing desktop or window-based applications. Core Java includes the following topics:

- Data types, variables, operators, and expressions
- Control flow statements, such as if-else, switch-case, loops, and break
- Arrays and strings
- Classes, objects, methods, and constructors
- Inheritance, polymorphism, abstraction, and encapsulation
- Interfaces and abstract classes
- Packages and access modifiers
- Exception handling and assertions
- Input and output streams
- Collections and generics
- Threads and concurrency
- Enums and annotations
- Lambda expressions and functional interfaces
- Java 8 features, such as streams, optional, and date-time API

Core Java is the foundation of Java programming, and it is essential to master it before moving on to advanced topics, such as Java EE (Enterprise Edition) or Java ME (Micro Edition). Core Java helps to understand the syntax, semantics, and logic of the Java language, and to write efficient, reliable, and maintainable code. Core Java also prepares the programmer for the various certification exams offered by Oracle and other vendors.

Introduction to Java

- Java is a high-level, object-oriented, platform-independent programming language that was developed by James Gosling at Sun Microsystems in

1991.

- Java is widely used for developing desktop, web, and mobile applications, as well as embedded systems and enterprise applications.
- Java follows the principle of “write once, run anywhere”, which means that the same Java code can run on different platforms, such as Windows, Linux, Mac OS, Android, etc., without requiring recompilation.
- Java is compiled into bytecode, which is a platform-independent representation of the program that can be executed by a Java Virtual Machine (JVM). The JVM is a software layer that interprets the bytecode and translates it into machine-specific instructions.
- Java supports multiple programming paradigms, such as imperative, declarative, functional, and concurrent programming. It also supports features such as inheritance, polymorphism, abstraction, encapsulation, exception handling, generics, annotations, lambda expressions, streams, etc.
- Java is designed to be simple, robust, secure, portable, dynamic, and efficient. It has a large and rich set of libraries and frameworks that provide various functionalities and services, such as networking, database, GUI, web, testing, etc.
- Java is one of the most popular and widely used programming languages in the world, according to various surveys and rankings. It is also one of the most taught languages in schools and universities.

Operator in Core Java

An operator in Core Java is a symbol that is used to perform some operations on data. Operators can manipulate the values of the operands, which are the variables or constants that the operators act on. For example, in the expression `a + b = c`, `a` and `b` are operands, and `+` is an operator that performs addition.

There are different types of operators in Core Java, such as:

- Arithmetic operators: They are used to perform simple arithmetic operations on primitive data types, such as addition, subtraction, multiplication, division, modulus, and exponentiation. For example, `a + b`, `a - b`, `a * b`, `a / b`, `a % b`, and `a ** b`.
- Unary operators: They are used to increment, decrement, or negate a value. They need only one operand. For example, `++a`, `--a`, and `-a`.
- Assignment operator: It is used to assign a value to a variable. For example, `a = b`.
- Compound assignment operators: They are used to perform an arithmetic operation and assign the result to the same variable. For example, `a += b` is equivalent to `a = a + b`.
- Relational operators: They are used to compare two values and return a boolean value (true or false) based on the result. For example, `a == b`, `a != b`, `a > b`, `a < b`, `a >= b`, and `a <= b`.
- Logical operators: They are used to combine two or more boolean expres-

sions and return a boolean value based on the logical rules. For example, `a && b`, `a || b`, and `!a`.

- Bitwise operators: They are used to perform operations on individual bits of an integer value. For example, `a & b`, `a | b`, `a ^ b`, `~a`, `a << b`, and `a >> b`.
- Conditional operator: It is a ternary operator that takes three operands and returns a value based on a condition. For example, `a ? b : c` returns `b` if `a` is true, and `c` if `a` is false.
- instanceof operator: It is used to check if an object is an instance of a specific class or interface. For example, `a instanceof B` returns true if `a` is an object of class `B` or a subclass of `B`.
- Type cast operator: It is used to convert a value from one data type to another. For example, `(int) a` converts `a` to an integer value.

Data type in Core Java

- A data type is a classification of data that specifies how the data is stored, manipulated, and interpreted by the compiler or the interpreter.
- Data types can be divided into two categories: primitive and reference.
- Primitive data types are the basic types of data that are built-in to the Java language. They are predefined and have fixed sizes and ranges. There are eight primitive data types in Java: byte, short, int, long, float, double, char, and boolean.
- Reference data types are the types of data that refer to objects or arrays. They are created by the programmer using class or interface definitions. Reference data types can store null values, which indicate the absence of an object or array. Reference data types can also be generic, which means they can hold values of different types depending on the type parameter specified.
- The following table summarizes the characteristics of the primitive data types in Java:

Data type	Size (in bits)	Range	Default value
byte	8	-128 to 127	0
short	16	-32768 to 32767	0
int	32	-2147483648 to 2147483647	0
long	64	-9223372036854775808 to 9223372036854775807	0L
float	32	1.4E-45 to 3.4028235E38	0.0f
double	64	4.9E-324 to 1.7976931348623157E308	0.0d

char | 16 | 0 to 65535 | '0000' |
boolean | 1 | true or false | false |

- The following table summarizes the characteristics of the reference data types in Java:

Data type	Size (in bytes)	Default value	Example
Class	Varies	null	String, Integer, Scanner, etc.
Interface	Varies	null	List, Set, Map, etc.
Array	Varies	null	int[], String[], Object[], etc.
Generic	Varies	null	List, Map<Integer, String>, etc.

Variable in Core Java

A variable in Java is a data container that stores the data values during the execution of a Java program. Every variable has a name and a data type that specifies the type and quantity of value it can hold. A variable is a name given to a memory location that can be used to store and retrieve data. Variables can be defined anywhere in the code, such as inside a class, inside a method, or as a method argument, and can have different modifiers. Depending on these conditions, variables in Java can be divided into four categories:

- **Instance variable:** A variable that is declared inside a class but outside a method is called an instance variable. It belongs to an object of the class and is created when the object is instantiated. Each object has its own copy of the instance variable and can access it using the dot operator. Instance variables can have any access modifier and can be initialized with a default or a user-defined value.
- **Static variable:** A variable that is declared inside a class but outside a method with the static keyword is called a static variable. It belongs to the class and is shared by all the objects of the class. It is created when the class is loaded and can be accessed using the class name and the dot operator. Static variables can have any access modifier and can be initialized with a default or a user-defined value.
- **Local variable:** A variable that is declared inside a method or a block of code is called a local variable. It is visible only within the scope of the method or the block and is destroyed when the method or the block exits. Local variables do not have any access modifier and must be initialized with a user-defined value before they can be used.
- **Parameter variable:** A variable that is declared as a method argument is called a parameter variable. It is used to pass data to the method when it is invoked. Parameter variables are local variables and have the same characteristics as local variables.

Arrays in Core Java

- An array is a collection of elements of the same data type that are stored in contiguous memory locations.
- An array can be declared using the syntax: `dataType[] arrayName;` or `dataType arrayName[];`

- An array can be initialized using the syntax: `arrayName = new dataType[size];` or `arrayName = {element1, element2, ..., elementN};`
- The size of an array is fixed and cannot be changed once it is created. The size can be obtained using the `length` attribute of the array: `arrayName.length`
- The elements of an array can be accessed using the index operator: `arrayName[index]`. The index starts from 0 and goes up to size-1.
- An array can be passed as an argument to a method using the syntax: `methodName(arrayName);`
- An array can be returned from a method using the syntax: `return arrayName;`
- An array can be multidimensional, meaning it can have more than one dimension. For example, a two-dimensional array can be declared using the syntax: `dataType[] [] arrayName;`
- A two-dimensional array can be initialized using the syntax: `arrayName = new dataType[rows][columns];` or `arrayName = {{element1, element2, ..., elementN}, {element1, element2, ..., elementN}, ..., {element1, element2, ..., elementN}};`
- The elements of a two-dimensional array can be accessed using the syntax: `arrayName[row][column]`. The row and column indices start from 0 and go up to rows-1 and columns-1 respectively.
- A two-dimensional array can be passed as an argument to a method using the syntax: `methodName(arrayName);`
- A two-dimensional array can be returned from a method using the syntax: `return arrayName;`
- An array can be nested inside another array, creating a jagged array. For example, a jagged array can be declared using the syntax: `dataType[] [] arrayName;`
- A jagged array can be initialized using the syntax: `arrayName = new dataType[size][];` or `arrayName = {{element1, element2, ..., elementN}, {element1, element2}, ..., {element1, element2, ..., elementN}};`
- The elements of a jagged array can be accessed using the syntax: `arrayName[index1][index2]`. The index1 starts from 0 and goes up to size-1. The index2 starts from 0 and goes up to the length of the subarray at index1.
- A jagged array can be passed as an argument to a method using the syntax: `methodName(arrayName);`
- A jagged array can be returned from a method using the syntax: `return arrayName;`

Methods & Classes in Core Java

- A method is a block of code that performs a specific task. A method can have parameters, which are values that are passed to the method when

it is invoked. A method can also return a value to the caller, or perform some action without returning anything.

- A class is a blueprint for creating objects. A class defines the properties and behaviors of the objects of that type. A class can have fields, which are variables that store data for each object, and methods, which are functions that operate on the object's fields or provide other services.
- To define a method in a class, use the following syntax:

```
modifier returnType methodName(parameterList) {  
    // method body  
}
```

- The modifier specifies the access level of the method, such as public, private, protected, or default. The returnType specifies the type of value that the method returns, or void if it does not return anything. The methodName is the name of the method, which must follow the Java naming conventions. The parameterList is a comma-separated list of parameters, each with a type and a name. The method body is a block of code that implements the logic of the method.
- To invoke a method in a class, use the following syntax:

```
objectName.methodName(argumentList);
```

- The objectName is the name of the object that the method belongs to, or the class name if the method is static. The methodName is the name of the method to be invoked. The argumentList is a comma-separated list of values that are passed to the method as parameters.
- To define a class in Java, use the following syntax:

```
modifier class ClassName {  
    // class body  
}
```

- The modifier specifies the access level of the class, such as public, private, protected, or default. The ClassName is the name of the class, which must follow the Java naming conventions. The class body is a block of code that contains the fields and methods of the class.
- To create an object of a class in Java, use the following syntax:

```
ClassName objectName = new ClassName(argumentList);
```

- The ClassName is the name of the class that the object belongs to. The objectName is the name of the object, which must follow the Java naming conventions. The argumentList is a comma-separated list of values that are passed to the constructor of the class, which is a special method that initializes the object's fields. The new keyword creates a new instance of the class and assigns it to the objectName.

Inheritance in Core Java

- Inheritance is a mechanism that allows one class to acquire the properties and behaviors of another class.
- The class that inherits from another class is called the subclass or derived class, and the class that is inherited from is called the superclass or base class.
- Inheritance is used to achieve code reusability, polymorphism, and abstraction in Java.
- In Java, inheritance is implemented using the `extends` keyword, which specifies that a subclass inherits from a superclass.
- For example, `class Dog extends Animal` means that the Dog class inherits from the Animal class.
- A subclass can access the public and protected members of its superclass, but not the private members.
- A subclass can also override the methods of its superclass, which means that it can provide a different implementation for a method that is already defined in the superclass.
- For example, the Dog class can override the `makeSound()` method of the Animal class and provide a different sound for dogs.
- In Java, a class can only inherit from one superclass, which means that Java does not support multiple inheritance. However, a class can implement multiple interfaces, which are similar to abstract classes that only contain abstract methods and constants.
- For example, `class Dog extends Animal implements Pet, Runnable` means that the Dog class inherits from the Animal class and implements the Pet and Runnable interfaces.

Package and Interface in Core Java

- A **package** is a group of classes, interfaces, sub-packages, and other types that are logically related and organized into a folder structure. A package helps to avoid naming conflicts, to find and use related types, and to reuse existing code. A package is created using the `package` keyword, and a package name usually follows the reverse domain name convention, such as `com.example.myapp`.
- An **interface** is a group of abstract methods and constants that define a contract or a behavior that a class can implement. An interface is created using the `interface` keyword, and an interface name usually follows the Pascal case convention, such as `MyInterface`. A class can implement one or more interfaces using the `implements` keyword, and must provide concrete implementations for all the abstract methods in the interfaces.
- Some of the benefits of using packages and interfaces in core Java are:
 - Packages and interfaces promote modularity and encapsulation, by separating the implementation details from the public interface of a type.
 - Packages and interfaces facilitate code reuse, by allowing existing types to be imported and implemented in different contexts.

- Packages and interfaces enhance code readability and maintainability, by grouping related types into meaningful categories and providing clear documentation for the expected behavior of a type.
- Packages and interfaces enable polymorphism and dynamic binding, by allowing a reference variable of an interface type to hold any object that implements that interface, and to invoke the appropriate method at run time.

Exception Handling in Core Java

- An exception is an abnormal or unexpected event that occurs during the execution of a program and disrupts its normal flow.
- Exception handling is a mechanism that allows a program to deal with exceptions gracefully and prevent the program from terminating abruptly.
- Exception handling in core Java involves four keywords: try, catch, throw and finally.
- The try block contains the code that may cause an exception. The catch block contains the code that handles the specific exception. The finally block contains the code that is always executed regardless of whether an exception occurs or not.
- The throw keyword is used to explicitly throw an exception from a method or a block of code. The throws keyword is used to declare that a method may throw one or more exceptions.
- Java provides two types of exceptions: checked and unchecked. Checked exceptions are those that are checked by the compiler at compile-time and must be handled or declared by the programmer. Unchecked exceptions are those that are not checked by the compiler and are usually caused by runtime errors or logical errors.
- Some of the common checked exceptions are IOException, FileNotFoundException, ClassNotFoundException, etc. Some of the common unchecked exceptions are ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc.
- Java also provides a hierarchy of exception classes that inherit from the java.lang.Throwable class. The Throwable class has two direct subclasses: java.lang.Exception and java.lang.Error. The Exception class is the superclass of all checked and unchecked exceptions. The Error class is the superclass of all errors that are usually fatal and should not be handled by the program.

Multithread programming in Core Java

- Multithread programming in Core Java is a process of executing two or more threads simultaneously to maximize the utilization of CPU .
- A thread is a lightweight sub-process, the smallest unit of processing. Threads can be created by using two mechanisms: extending the Thread class or implementing the Runnable interface.

- Threads can have different states, such as new, runnable, running, waiting, timed waiting, blocked, or terminated. The Thread class provides methods to control the thread's behavior, such as start, sleep, join, interrupt, yield, etc.
- Threads can communicate with each other by using methods like wait, notify, and notifyAll, which are defined in the Object class. These methods are used to implement synchronization among threads, which is essential to avoid data inconsistency and deadlock situations.
- Multithreading can improve the performance and responsiveness of an application, but it also introduces some challenges, such as concurrency issues, memory management, thread safety, etc. Therefore, it is important to use proper design patterns and best practices when developing multithreaded applications in Java.

I/O in Core Java

- I/O stands for Input and Output, which are the basic operations of any program that interacts with external sources, such as files, networks, databases, etc.
- Core Java provides a rich set of classes and interfaces in the `java.io` and `java.nio` packages to support various I/O operations.
- The main concept behind I/O in Core Java is the **stream**, which is an abstract representation of a sequence of bytes that can be read from or written to a source or a destination.
- There are two types of streams in Core Java: **byte streams** and **character streams**. Byte streams deal with raw binary data, while character streams deal with text data encoded in a specific charset.
- Byte streams are subclasses of `InputStream` and `OutputStream`, while character streams are subclasses of `Reader` and `Writer`.
- Some of the common byte stream classes are `FileInputStream`, `FileOutputStream`, `BufferedInputStream`, `BufferedOutputStream`, `DataInputStream`, `DataOutputStream`, etc.
- Some of the common character stream classes are `FileReader`, `FileWriter`, `BufferedReader`, `BufferedWriter`, `InputStreamReader`, `OutputStreamWriter`, etc.
- Core Java also supports **serialization**, which is the process of converting an object into a stream of bytes that can be stored or transmitted, and **deserialization**, which is the reverse process of reconstructing an object from a stream of bytes.
- Serialization and deserialization are implemented by the `ObjectInputStream` and `ObjectOutputStream` classes, which extend `InputStream` and `OutputStream`, respectively.
- To make an object serializable, it must implement the `Serializable` interface, which is a marker interface with no methods.
- Core Java also provides the `java.nio` package, which stands for New I/O or Non-blocking I/O, which is an alternative way of performing I/O

operations using **channels** and **buffers**.

- A channel is a connection to a source or a destination of data, such as a file, a socket, a pipe, etc. A buffer is a container for a fixed amount of data of a specific type, such as a byte, a char, an int, etc.
- Channels and buffers support both blocking and non-blocking modes of operation, which means that they can either wait for the data to be available or return immediately with whatever data is available.
- Some of the common channel classes are `FileChannel`, `SocketChannel`, `ServerSocketChannel`, `DatagramChannel`, etc.
- Some of the common buffer classes are `ByteBuffer`, `CharBuffer`, `IntBuffer`, etc.

Java Applet in Core Java

- A Java applet is a small application written in the Java programming language that can be transferred via the Internet and run by a Java-compatible web browser .
- The main difference between Java-based applications and applets is that applets are typically executed in an `AppletViewer` or Java-compatible web browser, while applications are executed by the Java Virtual Machine (JVM) directly.
- Java applets can provide web applications with interactive features that cannot be provided by HTML, such as graphics, animations, games, calculators, etc .
- To create a Java applet, one needs to extend the `java.applet.Applet` class and override the `init()`, `start()`, `stop()`, and `destroy()` methods. These methods are invoked by the browser or the `AppletViewer` to manage the applet's life cycle.
- To embed a Java applet in an HTML page, one needs to use the tag or the tag with the appropriate attributes, such as `code`, `width`, `height`, `archive`, etc. The `code` attribute specifies the name of the applet class file, and the `archive` attribute specifies the name of the JAR file that contains the applet and its resources.
- Java applets have some limitations and security restrictions, such as they cannot access the local file system, the network, or the clipboard of the user's computer, unless they are signed by a trusted authority. They also require the user to have a Java plug-in installed and enabled in the browser, which may not be supported by some browsers or platforms .
- Java applets are considered obsolete and deprecated since Java 9, and are no longer supported by most browsers. They have been replaced by other technologies, such as Java Web Start, JavaFX, HTML5, JavaScript, etc.

String handling in Core Java

- A string is a sequence of characters that can be manipulated by various methods and operators.

- In Java, strings are objects of the class `java.lang.String`, which is immutable, meaning that its value cannot be changed once created.
- To create a string object, we can use one of the following ways:
 - String literals: `String s = "Hello";`
 - String constructors: `String s = new String("Hello");`
 - String methods: `String s = s1.concat(s2);`
 - String pool: A memory area where all the string literals are stored and reused to save memory and improve performance.
- Some of the common methods and operators for string manipulation are:
 - `length()`: Returns the number of characters in the string.
 - `charAt(int index)`: Returns the character at the specified index in the string.
 - `equals(Object obj)`: Compares the string with another object for equality.
 - `equalsIgnoreCase(String anotherString)`: Compares the string with another string for equality, ignoring case differences.
 - `compareTo(String anotherString)`: Compares the string with another string lexicographically, returning a negative, zero, or positive value depending on the order.
 - `concat(String str)`: Returns a new string that is the concatenation of the string and the argument string.
 - `substring(int beginIndex, int endIndex)`: Returns a new string that is a substring of the string, starting from the begin index and ending before the end index.
 - `toLowerCase()`: Returns a new string that is the lowercase version of the string.
 - `toUpperCase()`: Returns a new string that is the uppercase version of the string.
 - `trim()`: Returns a new string that is the string with leading and trailing whitespace removed.
 - `indexOf(String str)`: Returns the index of the first occurrence of the argument string in the string, or -1 if not found.
 - `lastIndexOf(String str)`: Returns the index of the last occurrence of the argument string in the string, or -1 if not found.
 - `replace(char oldChar, char newChar)`: Returns a new string that is the string with all occurrences of the old character replaced by the new character.
 - `split(String regex)`: Returns an array of strings that are the result of splitting the string around matches of the argument regular expression.
 - `+`: The concatenation operator that appends the argument string to the string.
 - `+=`: The assignment operator that appends the argument string to the string and assigns the result to the string.

Event handling in Core Java

- An event is an action or occurrence that happens during the execution of a program, such as clicking a button, typing a key, moving a mouse, etc.
- Event handling is the process of responding to events by defining the logic or behavior that should be executed when an event occurs.
- Event handling in Core Java involves the following components:
 - Event sources: The objects that generate events, such as buttons, text fields, menus, etc.
 - Event listeners: The objects that receive events and perform the appropriate actions, such as implementing an interface or extending a class that defines the event handling methods.
 - Event objects: The objects that encapsulate the information about an event, such as the source, the type, the time, the coordinates, etc.
- Event handling in Core Java follows the delegation model, which means that the event source delegates the responsibility of handling the event to a separate event listener object.
- To handle an event in Core Java, the following steps are required:
 - Create an event source object and register one or more event listeners to it using the `addXXXListener()` method, where `XXX` is the type of the event, such as `Action`, `Mouse`, `Key`, etc.
 - Implement the event listener interface or extend the event adapter class that corresponds to the type of the event, and override the event handling methods that are invoked when the event occurs.
 - Write the logic or behavior that should be executed inside the event handling methods, using the event object as a parameter to access the information about the event.

Introduction to AWT in Core Java

- AWT stands for Abstract Window Toolkit, which is an API that provides classes and interfaces for creating graphical user interfaces (GUIs) and drawing graphics and images in Java .
- AWT is part of the Java Foundation Classes (JFC), which also includes Swing, Java 2D, and Accessibility.
- AWT components are platform-dependent, meaning that they are displayed according to the native look and feel of the operating system .
- AWT components are also heavyweight, meaning that they use the resources of the operating system, such as windows, fonts, and colors .
- AWT provides a hierarchy of classes and interfaces for creating and managing GUI components, such as buttons, text fields, labels, menus, dialogs, frames, etc .
- The root class of the AWT hierarchy is the `Component` class, which represents any GUI element that can be displayed on the screen and that can interact with the user .
- The `Component` class has two subclasses: `Container` and `MenuComponent`.

A Container is a component that can contain other components, such as a Panel, a Window, or a Frame. A MenuComponent is a component that represents a menu item, such as a MenuItem, a CheckboxMenuItem, or a MenuBar .

- A Window is a container that has no border, title bar, or menu bar. It is used to create pop-up windows, such as dialogs or message boxes. A Frame is a window that has a border, title bar, and menu bar. It is used to create the main window of an application .
- A Panel is a container that can be placed inside another container, such as a Frame or a Window. It is used to group and organize other components, such as buttons, labels, or text fields .
- AWT also provides classes and interfaces for handling events, such as mouse clicks, keyboard inputs, or window closing. The main interface for handling events is the ActionListener interface, which has several subinterfaces for different types of events, such as ActionListener, MouseListener, WindowListener, etc .
- AWT also provides classes and interfaces for drawing graphics and images, such as shapes, colors, fonts, or images. The main class for drawing graphics is the Graphics class, which provides methods for drawing lines, rectangles, ovals, polygons, text, or images. The Graphics class also has a subclass called Graphics2D, which provides more advanced features, such as transformations, gradients, or antialiasing .
- AWT also provides classes and interfaces for managing the layout of components, such as how they are arranged and sized within a container. The main interface for managing layout is the LayoutManager interface, which has several implementations, such as BorderLayout, GridLayout, FlowLayout, or BoxLayout .

AWT controls

- AWT stands for Abstract Window Toolkit, which is a set of APIs for creating graphical user interfaces (GUIs) in Java .
- AWT controls are components that allow a user to interact with the GUI in various ways, such as entering text, selecting options, clicking buttons, etc .
- AWT controls are also called heavy-weight components, because they rely on the native operating system (OS) for their appearance and functionality .
- The java.awt package provides classes for AWT controls, such as Label, Button, Checkbox, Choice, List, Scrollbar, TextField, TextArea, etc .
- Some of the commonly used AWT controls are:
 - Label: A component for displaying text in a container.
 - Button: A component for triggering an action when clicked.

- Checkbox: A component for selecting or deselecting an option in a binary state.
 - Choice: A component for selecting one option from a drop-down list.
 - List: A component for displaying and selecting multiple items from a scrollable list.
 - Scrollbar: A component for adjusting the visible portion of a container or a component.
 - TextField: A component for entering and editing a single line of text.
 - TextArea: A component for entering and editing multiple lines of text.
- To use AWT controls, you need to import the `java.awt` package, create an instance of the desired control class, and add it to a container, such as a `Frame` or a `Panel`.
 - For example, the following code snippet creates a `Label`, a `Button`, and a `TextField`, and adds them to a `Frame`:

```
import java.awt.*;
public class AWTExample {
    public static void main(String[] args) {
        // Create a frame
        Frame frame = new Frame("AWT Example");
        // Set the layout manager
        frame.setLayout(new FlowLayout());
        // Create a label
        Label label = new Label("Enter your name:");
        // Create a button
        Button button = new Button("Submit");
        // Create a text field
        TextField textField = new TextField(20);
        // Add the components to the frame
        frame.add(label);
        frame.add(textField);
        frame.add(button);
        // Set the size and visibility of the frame
        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}
```

Layout managers in AWT

- Layout managers are objects that control the size and position of the components in a container.
- Layout managers are useful for creating responsive and consistent user interfaces that can adapt to different screen sizes and resolutions.

- AWT provides five predefined layout managers that can be imported from the `java.awt` package: `FlowLayout`, `BorderLayout`, `GridLayout`, `CardLayout`, and `GridBagLayout`.
- Each layout manager has its own rules and constraints for arranging the components in a container.
- The default layout manager for applets and panels is `FlowLayout`, which places the components from left to right and wraps to the next line when the horizontal space is exhausted.
- `BorderLayout` divides the container into five regions: `NORTH`, `SOUTH`, `EAST`, `WEST`, and `CENTER`. Each region can contain only one component, and the `CENTER` region expands to fill the remaining space.
- `GridLayout` arranges the components in a grid of rows and columns, with equal size and spacing. The number of rows and columns can be specified in the constructor, or determined by the container size.
- `CardLayout` manages a stack of components, where only one component is visible at a time. The components can be switched by calling the `next`, `previous`, `first`, `last`, or `show` methods of the layout manager.
- `GridBagLayout` is the most flexible and complex layout manager, which allows the components to span multiple rows and columns, have different alignments, weights, and insets. The layout constraints for each component are specified by a `GridBagConstraints` object.
- In addition to the predefined layout managers, it is possible to create custom layout managers by implementing the `LayoutManager` interface, which defines four abstract methods: `addLayoutComponent`, `removeLayoutComponent`, `preferredLayoutSize`, and `layoutContainer`.

Unit 2 - Web Page Designing

- Web page designing is the process of creating and arranging the content of a web page, such as text, images, links, videos, etc.
- Web page designing involves two aspects: web design and web development.
- Web design is the visual and aesthetic aspect of a web page, such as layout, color scheme, typography, etc. Web design is usually done using tools like HTML, CSS, and JavaScript.
- Web development is the functional and interactive aspect of a web page, such as navigation, user input, data processing, etc. Web development is usually done using tools like PHP, Python, Ruby, etc.
- Web page designing follows some principles and guidelines, such as:
 - Consistency: The web page should have a consistent look and feel throughout, such as using the same fonts, colors, and layout.
 - Clarity: The web page should have a clear and concise content, such as using headings, bullet points, and white space.
 - Accessibility: The web page should be accessible to all users, such as using alt text for images, captions for videos, and keyboard navigation.

- Responsiveness: The web page should adapt to different screen sizes and devices, such as using media queries, flexbox, and grid.
- Usability: The web page should be easy to use and navigate, such as using buttons, menus, and breadcrumbs.
- SEO: The web page should be optimized for search engines, such as using keywords, meta tags, and sitemaps.

HTML in Web Page Designing

- HTML stands for HyperText Markup Language. It is the code that is used to structure a web page and its content.
- HTML uses different tags, elements, images and some latest components to make web pages more attractive and user-friendly.
- HTML tags are keywords that are enclosed in angle brackets (< and >). They indicate the start and end of an HTML element. For example, `<p>` and `</p>` are the tags for a paragraph element.
- HTML elements are the building blocks of a web page. They consist of a start tag, an end tag, and the content in between. Some elements have attributes, which are additional information that modify the element. For example, `` is an image element with two attributes: `src` and `alt`.
- HTML can be written by using any text editor, such as Notepad, TextEdit, or Brackets. The HTML code should be saved with a `.html` or `.htm` extension. To view the web page, the HTML file should be opened in a web browser, such as Chrome, Firefox, or Safari.
- HTML can be combined with CSS (Cascading Style Sheets) to change the look and feel of the web page. CSS is a language that defines how HTML elements are displayed, such as the color, font, layout, and animation. CSS can be written in a separate file and linked to the HTML file, or embedded in the HTML file using the `<style>` tag.
- HTML can also be made responsive, which means that the web page can automatically adjust to different screen sizes and devices. Responsive web design is achieved by using HTML and CSS to resize, hide, shrink, or enlarge the web page elements. One of the key techniques for responsive web design is to use the `<meta viewport>` tag to set the viewport, which is the area of the web page that the user can see.

List in Web Page Designing

A list is a way of presenting related items or data in a web page. Lists can help users to scan, compare, and find information easily. Lists can also enhance the visual appeal and readability of a web page. There are different types of lists that can be used in web page designing, such as:

- **Ordered lists:** These are lists that use numbers, letters, or other symbols to indicate the order or sequence of the items. Ordered lists are useful for showing steps, rankings, priorities, or chronological events. For example, an ordered list can show the steps to bake a cake, the top 10 movies of the year, or the history of a company. To create an ordered list in HTML, use the `<ol>` tag and the `<li>` tag for each list item. You can also use the `type` attribute to specify the style of the numbering, such as `1`, `a`, `i`, etc.
- **Unordered lists:** These are lists that use bullets, dashes, or other symbols to mark the items. Unordered lists are useful for showing categories, features, options, or general information. For example, an unordered list can show the ingredients of a recipe, the benefits of a product, or the services of a company. To create an unordered list in HTML, use the `<ul>` tag and the `<li>` tag for each list item. You can also use the `type` attribute to specify the style of the bullet, such as `disc`, `circle`, `square`, etc.
- **Definition lists:** These are lists that show the terms and their definitions or descriptions. Definition lists are useful for showing glossaries, acronyms, or key concepts. For example, a definition list can show the meaning of some technical terms, the abbreviations of some organizations, or the main ideas of a topic. To create a definition list in HTML, use the `<dl>` tag and the `<dt>` tag for each term and the `<dd>` tag for each definition or description.

Some tips for creating an excellent list in web page designing are:

- Keep the items logical, actionable, and consistent. The items should be related to each other and to the purpose of the list. They should also be clear, concise, and easy to understand. If the items are actions or tasks, they should use the same verb tense and format. If the items are information or data, they should use the same style and format.
- Focus on the visuals, alignment, spacing, and other details. The list should be attractive and eye-catching, but not distracting or cluttered. The list should also be aligned and spaced properly, with enough white space and contrast. The list should also use appropriate fonts, colors, and icons to enhance the readability and aesthetics.
- Add interactions, animations, or transitions, if necessary. The list can be more dynamic and engaging by adding some interactivity, such as hover effects, click events, drag and drop, etc. The list can also use some animations or transitions to show changes, progress, or feedback. For example, the list can fade in or out, slide, or expand when the user interacts with it.
- Show or hide lists, if necessary. The list can be shown or hidden depending on the context, the user's preference, or the device's screen size. For example, the list can be collapsed or expanded by clicking a button, a link, or an icon. The list can also be responsive and adaptive to different

screen sizes and resolutions, using media queries, breakpoints, or other techniques.

- Choose the right tools. The list can be created and designed using various tools, such as HTML, CSS, JavaScript, jQuery, Bootstrap, etc. The choice of the tool depends on the complexity, functionality, and style of the list. The tool should also be compatible with different browsers, devices, and platforms. The tool should also be easy to use, maintain, and update.

Table in Web Page Designing

- A table is a way of displaying and describing data in the form of rows and columns.
- A table can be used for various purposes, such as presenting tabular data, creating page layouts, or adding visual effects.
- A table is created using the `<table>` tag in HTML, which contains one or more `<tr>` tags for table rows, and one or more `<td>` tags for table cells.
- A table can also have a `<thead>` tag for table header, a `<tbody>` tag for table body, and a `<tfoot>` tag for table footer.
- A table can also have a `<caption>` tag for table caption, a `<colgroup>` tag for table column group, and a `<col>` tag for table column.
- A table can be styled using CSS properties, such as `border`, `padding`, `margin`, `background`, `color`, `font`, `text-align`, `vertical-align`, etc.
- A table can also use CSS pseudo-classes, such as `:hover`, `:active`, `:first-child`, `:last-child`, etc., to create dynamic effects.
- A table can also use CSS pseudo-elements, such as `::before`, `::after`, `::first-line`, `::first-letter`, etc., to add content or decoration.
- A table can also use CSS selectors, such as `#id`, `.class`, `[attribute]`, `:nth-child()`, `:not()`, etc., to target specific elements or groups of elements.
- A table can also use CSS media queries, such as `@media`, to adapt to different screen sizes or devices.
- A table can also use CSS flexbox or grid layout, to create responsive and flexible layouts.

Images in Web Page Designing

Images are an essential element of web page designing, as they can enhance the appearance, usability, and functionality of a website. Images can convey information, create emotions, attract attention, and support branding. However, images also have some drawbacks, such as increasing the loading time, requiring more bandwidth, and affecting the accessibility of a website. Therefore, web designers need to follow some best practices when using images in web page designing, such as:

- Choosing relevant images that match the content and purpose of the web page. Images should not be used randomly or excessively, as they can

distract or confuse the users. Images should also be consistent with the style and tone of the website.

- Optimizing images for different devices and screen sizes. Images should be responsive, meaning that they can adapt to different resolutions and orientations without losing quality or clarity. Images should also have appropriate file formats, dimensions, and compression levels, to reduce the file size and loading time.
- Providing alternative text for images that are not decorative. Alternative text, or alt text, is a brief description of the image that is displayed when the image cannot be loaded or viewed by the users. Alt text helps users with visual impairments or screen readers to understand the content and context of the image. Alt text also improves the search engine optimization (SEO) of the website, as it helps the search engines to index and rank the images.
- Using images to support the layout and navigation of the web page. Images can be used to create visual hierarchy, contrast, balance, and harmony in the web page design. Images can also be used to create buttons, icons, logos, and other graphical elements that help the users to interact with the website.

Frames in Web Page Designing

- Frames are a feature of HTML that allows you to control the layout of a website in the user's browser.
- Frames allow you to divide the browser window into rectangular sections that can be treated as if they were separate browser windows.
- Each frame can be scrolled and resized, and loaded with different web pages.
- To use frames on a page, you use the `<frameset>` tag instead of the `<body>` tag.
- The `<frameset>` tag defines how to divide the window into frames.
- The `rows` attribute of the `<frameset>` tag defines horizontal frames and the `cols` attribute defines vertical frames.
- Each frame is defined by a `<frame>` tag inside the `<frameset>` tag.
- The `<frame>` tag has a `src` attribute that specifies the URL of the web page to load in the frame.
- A frame on a frames page can also point to another frames page, creating a nested frameset.
- Frames can be useful for creating consistent navigation menus, headers, footers, or sidebars that remain visible while the main content changes .
- However, frames also have some disadvantages, such as:
 - They can disrupt the flow of the web and make it harder to bookmark, link, or print specific pages.
 - They can cause accessibility problems for users with screen readers, text browsers, or low bandwidth.
 - They can create maintenance issues and increase the complexity of

the website's code.

- Frames are obsolete in HTML5 and are not supported by some modern browsers.
- To create responsive and flexible layouts, it is recommended to use other techniques, such as CSS grid, flexbox, or media queries.

Forms in Web Page Designing

- Forms are web pages that allow users to enter and submit data to a web server using interactive controls.
- Forms can be used for various purposes, such as logging in, registering, contacting, ordering, searching, etc.
- Forms consist of the following components:
 - Structure: The order, layout, and grouping of the input fields and labels.
 - Input fields: The elements that accept user input, such as text boxes, checkboxes, radio buttons, sliders, etc.
 - Labels: The text that describes the purpose and format of the input fields.
 - Feedback: The messages that inform the user about the status, errors, or results of the form submission.
 - Actions: The buttons or links that allow the user to submit, reset, or cancel the form.
- Forms should follow the best practices of web design, such as :
 - Aligning the input fields and labels in a consistent and logical way.
 - Using clear and concise labels that match the user's expectations and vocabulary.
 - Providing helpful feedback and error messages that guide the user to complete the form successfully.
 - Using appropriate input types and attributes that match the data format and validation rules.
 - Minimizing the number of input fields and steps required to complete the form.
 - Using visual cues and affordances that indicate the required, optional, and disabled fields.
 - Using contrast, color, and typography to enhance the readability and usability of the form.
 - Testing the form on different devices, browsers, and screen sizes to ensure its functionality and accessibility.

CSS in Web Page Designing

CSS stands for Cascading Style Sheets. It is a language that is used to style and layout web pages. CSS can control various aspects of the appearance and behavior of a web page, such as:

- The color, font, size, and alignment of text

- The spacing, padding, margin, and border of elements
- The position, size, and shape of elements
- The background image, color, and gradient of elements
- The animation, transition, and transformation of elements
- The responsiveness and adaptability of elements to different screen sizes and devices

CSS works by applying rules to elements on a web page. A rule consists of a selector and a declaration block. A selector is a way of identifying which element or elements the rule applies to. A declaration block is a set of properties and values that define how the selected element or elements should look or behave. For example, the following rule sets the color of all paragraphs to blue and the font size to 18 pixels:

```
p {
  color: blue;
  font-size: 18px;
}
```

CSS can be written in three different ways:

- Inline CSS: The style attribute is used to apply CSS directly to an HTML element. This method is not recommended as it makes the HTML code messy and hard to maintain.
- Internal CSS: The style element is used to write CSS inside the head section of an HTML document. This method is useful for applying CSS to a single web page.
- External CSS: The link element is used to link an external CSS file to an HTML document. This method is preferred as it allows the CSS code to be reused and updated across multiple web pages.

CSS is a powerful and flexible tool for web page designing. It can help to create attractive, consistent, and user-friendly web pages that can adapt to different devices and browsers. CSS is also easy to learn and use, as it follows a simple and logical syntax. CSS is widely supported by all modern web browsers and is an essential skill for web developers and designers.

Document type definition in Web Page Designing

- A document type definition (DTD) is a set of rules that defines the structure and syntax of a markup language, such as HTML or XML.
- A DTD specifies what elements, attributes, entities, and notations are allowed in a document, and how they are organized and nested.
- A DTD can be declared either internally or externally to a document. An internal DTD is embedded within the document itself, while an external DTD is referenced by a URL or a file path.
- A DTD can be used to validate a document against the rules of the markup language, and to ensure that the document is well-formed and consistent.

- A DTD can also be used to provide default values for attributes, declare entities for special characters or common phrases, and define notations for non-XML data, such as images or multimedia.
- A DTD can be written in two syntaxes: SGML or XML. SGML is the older and more complex syntax, while XML is the newer and simpler syntax. XML DTDs are more widely used and supported than SGML DTDs.
- A DTD starts with a document type declaration, which identifies the name of the document type and the location of the DTD. For example, the following declaration indicates that the document is an HTML document and the DTD is located at the URL <http://www.w3.org/TR/html4/strict.dtd>:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
```

- A DTD consists of a series of declarations that define the elements, attributes, entities, and notations of the document type. For example, the following declarations define an element called title, an attribute called lang, an entity called copy, and a notation called gif:

```
<!ELEMENT title (#PCDATA)>
<!ATTLIST title lang CDATA #IMPLIED>
<!ENTITY copy "&copy;">
<!NOTATION gif SYSTEM "image/gif">
```

- A DTD can also include comments, conditional sections, and parameter entities. Comments are used to add notes or explanations to the DTD, and start and end with `<!--` and `-->`. Conditional sections are used to include or exclude parts of the DTD based on certain conditions, and start and end with `<![` and `]>`. Parameter entities are used to define reusable chunks of DTD code, and start and end with `%` and `;`. For example, the following DTD uses a comment, a conditional section, and a parameter entity:

```
<!-- This is a comment -->
<!ENTITY % heading "h1|h2|h3|h4|h5|h6">
<![INCLUDE [
<!ELEMENT %heading; (#PCDATA)>
<!ATTLIST %heading; align (left|center|right) #IMPLIED>
]]>
```

XML in Web Page Designing

XML stands for **Extensible Markup Language**. It is a markup language that allows you to define your own tags to describe and structure data. XML is widely used for web development, as it can store and transport data across different platforms and applications. Some of the benefits of using XML in web page designing are:

- **Domain-specific vocabulary:** You can create custom tags that suit your specific needs and domain. For example, you can use XML to describe

books, products, recipes, etc. with tags that are meaningful and relevant to your data. This makes your data more understandable and reusable.

- **Separation of content and presentation:** You can use XML to store the content of your web pages, and use other technologies, such as CSS, XSLT, or JavaScript, to define how the content should be displayed. This allows you to change the appearance of your web pages without affecting the underlying data, and vice versa.
- **Interoperability and compatibility:** You can use XML to exchange data between different applications and systems, as long as they can parse and process XML. XML is a standard format that is supported by many tools and languages, such as Java, Python, PHP, etc. XML is also compatible with Unicode, which means it can represent data in any human language.
- **Validation and verification:** You can use XML schemas or DTDs to define the rules and constraints for your XML data. This allows you to check the validity and accuracy of your data, and prevent errors and inconsistencies. You can also use XML namespaces to avoid conflicts and ambiguities between different XML vocabularies.

DTD in Web Page Designing

- DTD stands for Document Type Definition .
- A DTD defines the structure and the legal elements and attributes of an XML document .
- A DTD can be declared internally or externally in an XML document.
- A DTD can be used to validate an XML document against a set of rules .
- A DTD can also be used to declare entities, notations, and processing instructions.
- In HTML 4.01, the DOCTYPE declaration refers to a DTD .
- The DOCTYPE declaration is an instruction to the web browser about what version of HTML the page is written in .
- The DOCTYPE declaration ensures that the web page is parsed the same way by different web browsers .
- There are three types of DTDs in HTML 4.01 :
 - Strict DTD: Used for web pages that exclude attributes and elements that W3C expects to phase out as CSS support grows .
 - Transitional DTD: Used for web pages that include attributes and elements that W3C expects to phase out as CSS support grows .
 - Frameset DTD: Used for web pages with frames .
- In HTML5, the DOCTYPE declaration is simplified and does not refer to a DTD.
- The DOCTYPE declaration in HTML5 is `<!DOCTYPE html>` and it tells the browser to render the page in standards mode.

XML Schemas in Web Page Designing

- XML stands for eXtensible Markup Language and is used to store and transport data.
- XML schemas are XML-based documents that describe and validate the structure and the content of XML data.
- XML schemas support data types, which make it easier to describe allowable document content, validate the correctness of data, and define data facets (restrictions on data).
- XML schemas can be built in-memory using the classes in the System.Xml.Schema namespace, which map to the structures defined in the World Wide Web Consortium (W3C) XML Schema Recommendation.
- XML schemas can contain different design patterns, such as Russian Doll, Salami Slice, Venetian Blind, and Garden of Eden, which vary according to the number of their global elements or types.
- A global element or type is a child of the schema element that contains a target namespace.
- XML schemas can be derived by restriction from other simple types (built-in or user-defined) or constructed as a list or union of other simple types.
- XML schemas can be used to create web pages that are well-formed, valid, and consistent with the data model and business rules of the application.

Object Models in Web Page Designing

- An object model is a visual representation of a system's objects, actions, and associated attributes.
- An object model can be used, in conjunction with a design system, to create a consistent and intuitive user interface for a web page.
- An object model can also be used to automate the testing of a web page using tools like Selenium, which is a framework for web browser automation.
- One of the common design patterns in web page automation is the Page Object Model (POM), which is used to create an object repository for all web UI elements.
- In POM, each web page is represented by a corresponding class, and each web element is represented by a variable of the class. The class also contains methods or functions to interact with the web elements.
- POM has several advantages, such as:
 - It improves the readability and maintainability of the test code by separating the test logic from the web element locators.
 - It reduces the code duplication and the risk of errors by reusing the same web elements and methods across different test cases.
 - It makes the test code more robust and adaptable to changes in the web page structure or layout by centralizing the web element locators in one place.
- POM can be implemented in different programming languages, such as

Java, Python, JavaScript, etc. using Selenium or other web automation frameworks .

- POM can be combined with other design patterns, such as ScreenPlay Model, Data-Driven Testing, Keyword-Driven Testing, etc. to enhance the test automation capabilities and performance.

Presenting and using XML in Web Page Designing

XML stands for eXtensible Markup Language. It is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable. XML was designed to store and transport data across the Internet. Unlike HTML, which has predefined tags for presentation, XML allows users to create their own tags to describe the structure and meaning of the data .

Some of the benefits of using XML in web page designing are:

- XML is platform-independent and can be used with any programming language or software.
- XML is self-describing and can be easily understood by humans and machines.
- XML can be validated using schemas or DTDs to ensure the data conforms to a specific structure and format.
- XML can be transformed using XSLT or CSS to display the data in different ways.
- XML can be queried using XPath or XQuery to extract specific information from the data.
- XML can be used to exchange data between different applications or web services using protocols such as SOAP or REST .

Some of the examples of using XML in web page designing are:

- RSS feeds: RSS is a format for syndicating web content such as news, blogs, or podcasts. RSS uses XML to describe the metadata and content of the web items.
- SVG: SVG is a format for creating vector graphics on the web. SVG uses XML to describe the shapes, colors, and transformations of the graphics.
- MathML: MathML is a format for displaying mathematical expressions on the web. MathML uses XML to describe the symbols, operators, and layout of the expressions.
- XHTML: XHTML is a version of HTML that follows the rules of XML. XHTML uses XML to describe the structure and presentation of the web pages .

Using XML Processors in Web Page Designing

- XML stands for eXtensible Markup Language. It is a markup language similar to HTML, but without predefined tags to use. Instead, you define

your own tags designed specifically for your needs. This is a powerful way to store data in a format that can be stored, searched, and shared .

- XML by itself is not enough to create a web page. It needs to be processed by an XML processor, which is a software that can read, validate, and manipulate XML documents. An XML processor can also transform XML documents into other formats, such as HTML, using a stylesheet language such as XSLT .
- An example of an XML processor is XSLTProcessor, which is a web API that allows you to apply an XSLT stylesheet to an XML document and obtain a transformed document. You can use XSLTProcessor in a browser to dynamically generate web pages from XML data.
- Using XML processors in web page designing has several advantages, such as:
 - Separation of content and presentation: You can store the data in XML and use different stylesheets to render it in different ways, such as HTML, PDF, or plain text. This makes it easier to maintain and update the content without affecting the presentation .
 - Customization and interactivity: You can use XML to create interactive pages that allow the customer to customize their preferences, such as language, layout, or theme. You can also use XML to create e-commerce applications that can handle transactions, orders, and inventory .
 - Portability and compatibility: You can use XML to exchange data between different platforms, applications, and devices. XML is a standard format that can be read and processed by any XML processor. XML also supports Unicode, which means it can handle any character set .
- Using XML processors in web page designing also has some challenges, such as:
 - Complexity and learning curve: XML is a verbose and complex language that requires a lot of syntax and rules. You also need to learn how to use XML processors and stylesheet languages, such as XSLT, which have their own syntax and rules. This can be daunting for beginners and require more time and effort .
 - Performance and compatibility issues: XML processing can be slow and resource-intensive, especially for large and complex documents. You also need to ensure that your XML processor and stylesheet are compatible with the browser and device that you are targeting. Not all browsers and devices support the same features and standards of XML processing .

DOM and SAX in Web Page Designing

- DOM stands for **Document Object Model**, which is a convention for representing and interacting with objects in HTML, XML, and other document formats.

- SAX stands for **Simple API for XML**, which is an event-driven, serial-access mechanism for accessing XML documents.
- The main differences between DOM and SAX are:
 - DOM creates a **tree structure** in memory that represents the entire document, while SAX **reads the document sequentially** and triggers events when encountering elements, attributes, or text.
 - DOM allows **random access** and **manipulation** of any part of the document, while SAX only allows **one-way traversal** and **read-only access** of the document.
 - DOM is suitable for **small to medium size** XML documents that need to be queried or modified in different ways, while SAX is suitable for **large or streaming** XML documents that only need to be processed once.
 - DOM is a **standard interface** that is implemented by many languages and libraries, while SAX is a **low-level API** that varies across different parsers.

Dynamic HTML in Web Page Designing

- Dynamic HTML (DHTML) is a term that refers to the combination of various web development technologies for creating dynamic and interactive web pages .
- Dynamic HTML can work with HTML, JavaScript, XML, and CSS to manipulate the HTML elements and their styles, events, and behaviors .
- Dynamic HTML can create effects such as animation, transitions, drag and drop, form validation, and user interaction that would otherwise be impossible or require server-side processing .
- Dynamic HTML can also make web pages responsive, which means they can automatically adjust to different screen sizes and devices.
- Dynamic HTML uses the Document Object Model (DOM), which is a standard interface that represents the structure and content of a web page as a tree of objects.
- Dynamic HTML can access and modify any part of the DOM using scripts, which are usually written in JavaScript.
- Dynamic HTML can also use Cascading Style Sheets (CSS), which are rules that define the appearance and layout of the HTML elements.
- Dynamic HTML can apply different styles to the same HTML element depending on various conditions, such as user actions, browser settings, or media queries .
- Dynamic HTML can also use Extensible HTML (XHTML), which is a stricter and cleaner version of HTML that follows the rules of XML.
- Dynamic HTML can make web pages more user-friendly, attractive, and engaging, but it can also pose some challenges, such as browser compatibility, accessibility, and security.

Unit 3 - Scripting

Scripting is a way of automating tasks or creating programs using a scripting language. A scripting language is a high-level programming language that is interpreted or executed by another program, such as a shell, a browser, or a web server. Scripting languages are often used for tasks that are repetitive, complex, or require user interaction.

Some of the advantages of scripting are:

- Scripting languages are usually easy to learn and write, as they have a simple syntax and less strict rules than compiled languages.
- Scripting languages are portable, as they can run on different platforms and environments without requiring compilation or installation.
- Scripting languages are flexible, as they can interact with other programs and data formats, and can be modified or extended at runtime.
- Scripting languages are expressive, as they can perform complex operations with fewer lines of code than compiled languages.

Some of the disadvantages of scripting are:

- Scripting languages are usually slower than compiled languages, as they have to be interpreted or executed by another program, which adds overhead and reduces performance.
- Scripting languages are less secure than compiled languages, as they can be easily modified or accessed by unauthorized users or malicious code.
- Scripting languages are less reliable than compiled languages, as they may have errors or bugs that are not detected until runtime, and may not handle exceptions or errors gracefully.

Some of the examples of scripting languages are:

- Shell scripting: A way of automating tasks or commands in a shell, such as Bash, PowerShell, or CMD. Shell scripts can manipulate files, directories, processes, and environment variables, and can invoke other programs or scripts.
- Python: A general-purpose, multi-paradigm scripting language that is widely used for data analysis, web development, machine learning, and automation. Python has a simple and elegant syntax, a large standard library, and many third-party modules and frameworks.
- JavaScript: A scripting language that is mainly used for creating dynamic and interactive web pages. JavaScript can run in a browser, a web server, or a standalone environment, and can manipulate the Document Object Model (DOM), the Hypertext Transfer Protocol (HTTP), and the JavaScript Object Notation (JSON).
- Ruby: A scripting language that is designed to be expressive, concise, and elegant. Ruby supports multiple programming paradigms, such as object-oriented, functional, and procedural, and has a rich set of built-in methods and libraries, as well as a popular web framework called Ruby on Rails.

- Perl: A scripting language that is known for its powerful text processing and regular expression capabilities. Perl can handle various data formats, such as XML, JSON, and CSV, and can interface with databases, web servers, and network protocols. Perl has a motto of “There’s more than one way to do it”, which reflects its flexibility and versatility.

JavaScript in Scripting

- JavaScript is a **lightweight, interpreted, or just-in-time compiled** programming language with **first-class functions**.
- JavaScript is most well-known as the **scripting language for Web pages**, but it can also be used in many non-browser environments, such as Node.js, Apache CouchDB and Adobe Acrobat.
- Scripting is a type of programming that allows you to **implement complex features** on web pages, such as displaying timely content updates, interactive maps, animated graphics, scrolling video jukeboxes, etc.
- JavaScript scripts are **interpreted by the browser** and run in the browser’s **execution environment** .
- JavaScript scripts can be **embedded** in HTML documents or **linked** to external files.
- JavaScript scripts can **access and manipulate** the HTML elements and CSS styles of the web page, as well as **respond to user events** such as clicks, mouse movements, keyboard inputs, etc.
- JavaScript scripts can also **communicate** with web servers, databases, web APIs, and other web services using various methods, such as XMLHttpRequest, Fetch, WebSocket, etc.
- JavaScript scripts can also **use libraries and frameworks** that provide additional features and functionalities, such as jQuery, React, Angular, Vue, etc.
- JavaScript is a **dynamic, weakly typed, prototype-based, and multi-paradigm** language, which means it can support different programming styles, such as imperative, declarative, functional, object-oriented, etc .
- JavaScript has a **standard specification** called ECMAScript, which defines the core features and syntax of the language. There are also various **extensions** and **dialects** of JavaScript, such as TypeScript, CoffeeScript, etc, that add or modify some aspects of the language .

Introduction to JavaScript

- JavaScript is a **scripting language** that is used to create and manage **dynamic web pages**, basically anything that moves on your screen without requiring you to refresh your browser .
- JavaScript can be written right in a web page’s HTML and run automatically as the page loads. Scripts are provided and executed as plain text. They don’t need special preparation or compilation to run.
- JavaScript is a **multi-paradigm** language, meaning that it supports dif-

ferent programming styles, such as **object-oriented**, **functional**, **imperative**, and **declarative**.

- JavaScript has **types** and **operators**, such as numbers, strings, booleans, arithmetic, comparison, and logical operators.
- JavaScript has **standard built-in objects**, such as **Array**, **Date**, **Math**, and **String**, and **methods** that can be used to manipulate them.
- JavaScript has a **syntax** that is based on the Java and C languages, but it is not a subset or a variant of them. It has its own unique features and rules .
- JavaScript follows the **ECMAScript** standard, which defines the core features and syntax of the language. The current version of the standard is **ECMAScript 2022**.
- JavaScript can be used for various purposes, such as **web development**, **game development**, **data analysis**, **machine learning**, and more. It can run on different platforms, such as **browsers**, **servers**, **mobile devices**, and **embedded systems** .

Documents in JavaScript

- A document in JavaScript is an object that represents a web page loaded in a browser and provides access to its content, such as HTML elements, text, images, etc.
- A document is also an instance of the Document interface, which defines the properties and methods that can be used to manipulate the document object.
- A document is part of the Document Object Model (DOM), which is a tree-like structure that represents the logical relationship between the document and its elements.
- A document can be accessed and modified by using the global document object, which is a property of the window object.
- Some of the common properties and methods of the document object are:
 - `document.body`: returns the body element of the document.
 - `document.title`: returns or sets the title of the document.
 - `document.getElementById(id)`: returns the element with the specified id attribute.
 - `document.createElement(tagName)`: creates a new element with the specified tag name.
 - `document.appendChild(child)`: appends a new child element to the document.
 - `document.querySelector(selector)`: returns the first element that matches the specified CSS selector.
 - `document.querySelectorAll(selector)`: returns a collection of all elements that match the specified CSS selector.

- `document.addEventListener(type, listener)`: registers an event listener for the specified event type on the document.
- `document.removeEventListener(type, listener)`: removes an event listener for the specified event type from the document.
- `document.write(text)`: writes the specified text to the document.
- `document.close()`: closes the document after writing.
- For more information on the document object and its properties and methods, please refer to the following sources:
 - <https://www.positioniseverything.net/javascript-document/>
 - https://www.tutorialspoint.com/javascript/javascript_html_dom.htm
 - https://www.w3schools.com/js/js_htmldom_document.asp
 - https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

Forms in JavaScript

- Forms are an essential part of HTML pages, and developers typically use JavaScript to elaborate on how they function.
- Forms allow users to enter and submit data to a server of some kind.
- To create a form in HTML, you use the `<form>` element with attributes such as `action`, `method`, and `id`.
- To reference the `<form>` element in JavaScript, you can use DOM selecting methods such as `getElementById()`.
- To access the form fields, you can use the `elements` property of the form object, which returns a collection of input elements.
- To validate the form data, you can use JavaScript to check the input values and display error messages if they are invalid.
- To submit the form data, you can use the `submit()` method of the form object, or attach an event listener to the submit button.
- To send the form data using JavaScript, you can use the `FormData` object, which can build form data for transmission or get the data within a form element.

Statements in JavaScript

- A statement is a piece of code that performs some action or produces some value.
- Statements can be simple or complex, depending on the number of sub-statements they contain.
- Statements are executed in order, from top to bottom, unless there is a control flow statement that changes the order of execution.
- Statements are usually terminated by a semicolon (;), but it is optional in most cases.
- Some examples of statements are:

```
// A simple statement that assigns a value to a variable
var x = 10;
```

```

// A complex statement that contains a conditional expression and a block of statements
if (x > 0) {
    console.log("x is positive");
    x--;
}

// A statement that defines a function
function add(a, b) {
    return a + b;
}

// A statement that calls a function
var sum = add(3, 4);

// A statement that creates an object
var person = {
    name: "Alice",
    age: 25
};

// A statement that accesses a property of an object
var name = person.name;

// A statement that loops through an array
for (var i = 0; i < array.length; i++) {
    console.log(array[i]);
}

```

Functions in JavaScript

- A function is a block of code that performs a specific task and can be reused throughout the program.
- A function can have zero or more parameters that are passed as values or references when the function is called.
- A function can return a value to the caller using the **return** statement, or return nothing (undefined) by default.
- A function can be defined in three ways: as a function declaration, a function expression, or an arrow function.
- A function declaration is a statement that starts with the **function** keyword, followed by the function name, the parameter list, and the function body.
- A function expression is an expression that assigns a function to a variable or a property of an object. The function name is optional and can be omitted.
- An arrow function is a concise syntax for writing function expressions that

uses the `=>` operator. It does not have its own `this`, `arguments`, `super`, or `new.target` keywords and cannot be used as constructors.

- A function can be called by using its name followed by the argument list in parentheses, or by using the `call`, `apply`, or `bind` methods of the function object.
- A function can be nested inside another function, creating a closure that can access the variables and parameters of the outer function.
- A function can be a first-class value that can be stored in variables, passed as arguments, returned from other functions, or assigned to properties of objects.

Objects in JavaScript

- Objects are variables that contain multiple data values, in the form of key-value pairs. The keys can be variables or functions, and are called properties or methods, respectively.
- Objects can be created using the `Object()` constructor or the object literal syntax, which uses curly braces `{}` to enclose the key-value pairs.
- For example, the following code creates an object named `person` with three properties: `name`, `age`, and `greet`.

```
// Using the Object() constructor
var person = new Object();
person.name = "Alice";
person.age = 25;
person.greet = function() {
  console.log("Hello, I am " + this.name);
};

// Using the object literal syntax
var person = {
  name: "Alice",
  age: 25,
  greet: function() {
    console.log("Hello, I am " + this.name);
  }
};
```

- To access or modify the properties or methods of an object, we can use the dot notation `.` or the bracket notation `[]`.
- For example, the following code prints the name and age of the `person` object, and invokes the `greet` method.

```
// Using the dot notation
console.log(person.name); // Alice
console.log(person.age); // 25
person.greet(); // Hello, I am Alice
```

```
// Using the bracket notation
console.log(person["name"]); // Alice
console.log(person["age"]); // 25
person["greet"](); // Hello, I am Alice
```

- Objects in JavaScript are dynamic, meaning that we can add, delete, or change their properties or methods at any time.
- For example, the following code adds a new property **gender** and deletes the **age** property from the **person** object.

```
// Adding a new property
person.gender = "female";
console.log(person.gender); // female
```

```
// Deleting a property
delete person.age;
console.log(person.age); // undefined
```

- Objects in JavaScript are also mutable, meaning that they can be modified by reference. This means that if we assign an object to another variable, both variables will point to the same object in memory, and any changes made to one will affect the other.
- For example, the following code assigns the **person** object to a new variable **anotherPerson**, and changes the **name** property of **anotherPerson**. This also changes the **name** property of **person**.

```
// Assigning an object to another variable
var anotherPerson = person;
console.log(anotherPerson.name); // Alice
```

```
// Changing the name property of anotherPerson
anotherPerson.name = "Bob";
console.log(anotherPerson.name); // Bob
console.log(person.name); // Bob
```

- Objects in JavaScript are instances of the **Object** type, which is the root of the prototype chain. This means that all objects inherit some properties and methods from the **Object.prototype** object, such as **toString()**, **hasOwnProperty()**, and **valueOf()**.
- We can also create our own custom objects by using constructor functions or classes, which allow us to define the properties and methods of the objects, and set their prototype to a specific object. This way, we can implement inheritance and polymorphism in JavaScript.

Introduction to AJAX in JavaScript

- AJAX stands for **A**synchronous **J**avaScript and **X**ML .

- AJAX is not a programming language, but a technique for creating dynamic and interactive web applications .
- AJAX uses a combination of the following technologies :
 - **JavaScript** to make requests and handle responses
 - **XMLHttpRequest** object to communicate with the web server
 - **XML**, **JSON**, or **plain text** to exchange data
 - **HTML** and **CSS** to display the data
 - **DOM** to manipulate the web page elements
- AJAX allows you to send and receive data **asynchronously** without reloading the web page .
- AJAX can improve the user experience and performance of web applications by reducing the bandwidth and latency.
- AJAX can also enable web applications to communicate with other web services and APIs.

Networking in Scripting

- Networking in scripting refers to the use of scripts to automate various tasks related to network administration, such as configuring, monitoring, troubleshooting, and securing network devices and services.
- Scripts are programs that are written in a scripting language, such as PowerShell, Python, Perl, or Bash, and can be executed by an interpreter without the need for compilation.
- Scripting can offer significant advantages for network administrators, such as:
 - Saving time and effort by performing complex and repetitive tasks with a single command or a scheduled script.
 - Increasing consistency and accuracy by reducing human errors and ensuring compliance with standards and policies.
 - Enhancing flexibility and scalability by adapting to changing network conditions and requirements.
 - Improving productivity and performance by optimizing network resources and processes.
- Scripting can also enable network administrators to leverage the capabilities of network programming, such as:
 - Communicating with network devices and services using various protocols, such as HTTP, TCP, UDP, and SNMP.
 - Manipulating and analyzing network data, such as packets, headers, payloads, and logs.
 - Integrating with other systems and applications, such as databases, web servers, cloud platforms, and DevOps tools.
 - Developing and testing custom network solutions, such as scripts, modules, libraries, and APIs.
- Scripting requires network administrators to have some basic knowledge of programming concepts, such as variables, data types, operators, control structures, functions, and objects.

- Scripting also requires network administrators to have some familiarity with the syntax and features of the scripting language they choose to use, as well as the network environment and tools they work with.
- Scripting can be learned through various resources, such as books, online courses, tutorials, blogs, forums, and documentation.

Internet Addressing in Networking

- Internet addressing is the process of assigning unique identifiers to devices on a network that communicate using the Internet Protocol (IP) .
- An internet address works like a postal address, allowing data to be routed to the chosen destination .
- Internet addresses are made up of a network address and a host (or local) address . This two-part address allows a sender to specify the network as well as a specific host on the network .
- A network address identifies a group of hosts that share a common network segment . A host address identifies a specific device within a network .
- Internet addresses can be of numeric type or symbolic type or both in some cases . For example, IP addresses are numeric, domain names are symbolic, and URLs are both .
- There are two versions of IP addresses: IPv4 and IPv6 . IPv4 addresses are 32 bits long and consist of four numbers separated by dots, such as 192.168.1.34 . IPv6 addresses are 128 bits long and consist of eight groups of four hexadecimal digits separated by colons, such as 2001:db8:0:1234:0:567:8:1 .
- Internet addresses are assigned by different authorities depending on the scope and level of the network . For example, the Internet Assigned Numbers Authority (IANA) allocates IP address blocks to regional internet registries (RIRs), which in turn allocate them to local internet registries (LIRs), such as internet service providers (ISPs), which then assign them to end users .
- Internet addresses can also be classified into different types based on their functions and roles . For example, there are special addresses for broadcasts, multicasts, anycasts, and local loopback . Broadcast addresses are used to send data to all hosts on a network, multicast addresses are used to send data to a group of hosts, anycast addresses are used to send data to the nearest host, and local loopback addresses are used to test the connectivity of a host with itself .

InetAddress in Networking

- The InetAddress class is used to encapsulate both the numerical IP address and the domain name for that address .
- An IP address is a unique numerical label assigned to a machine in a network. It can be either 32-bit (IPv4) or 128-bit (IPv6) .
- An instance of InetAddress represents an IP address with its correspond-

ing host name, which is more convenient and understandable than its IP address .

- The `InetAddress` class can handle both IPv4 and IPv6 addresses .
- There are two types of addresses: Unicast and Multicast .
 - Unicast addresses are used to identify a single host in a network.
 - Multicast addresses are used to identify a group of hosts that can receive the same message.
- The `InetAddress` class provides methods to create, compare, and resolve IP addresses and host names .
 - To create an `InetAddress` object, you can use the static methods `getByName`, `getByAddress`, `getAllByName`, or `getLoopbackAddress` of the `InetAddress` class .
 - To compare two `InetAddress` objects, you can use the `equals`, `isAnyLocalAddress`, `isLoopbackAddress`, `isLinkLocalAddress`, `isSiteLocalAddress`, `isMulticastAddress`, or `isReachable` methods of the `InetAddress` class .
 - To resolve an IP address to a host name or vice versa, you can use the `getHostName`, `getHostAddress`, or `getCanonicalHostName` methods of the `InetAddress` class .
- The `InetAddress` class is an abstract class that has two subclasses: `Inet4Address` and `Inet6Address`, which represent IPv4 and IPv6 addresses respectively .
- The `InetAddress` class is immutable, meaning that its state cannot be changed after it is created .

Factory Methods in Networking

- Factory methods are static methods in a class that return an object of that class or its subclasses.
- Factory methods are used to create instances for classes without exposing the details of the class module to the user.
- Factory methods can be useful in networking applications, such as creating objects of the `InetAddress` class in Java, which represents an IP address.
- The `InetAddress` class has no visible constructors, so factory methods are used to create objects of this class.
- Some of the factory methods in the `InetAddress` class are:
 - `getByName(String host)`: Returns an `InetAddress` object for the given host name.
 - `getByAddress(byte[] addr)`: Returns an `InetAddress` object for the given raw IP address in a byte array.
 - `getAllByName(String host)`: Returns an array of `InetAddress` objects for all the IP addresses of the given host name.
 - `getLocalHost()`: Returns the `InetAddress` object of the local host.

- Factory methods can also be used to create objects of other networking classes, such as `Socket`, `ServerSocket`, `DatagramSocket`, etc.
- Factory methods can help to abstract the creation of objects from the client code, and allow for flexibility and extensibility in the design of networking applications.

Instance Methods in Networking

- Instance methods are methods that are defined on an instance of a class, such as a socket, a server, or a client.
- Instance methods can be used to perform various operations on the instance, such as sending and receiving data, closing the connection, or listening for incoming requests.
- Instance methods are usually invoked by using the dot notation, such as `socket.send(data)` or `server.accept()`.
- Some common instance methods in networking are:
 - `socket.bind(address)`: This method binds the socket to a specific address and port. The address can be a hostname, an IP address, or an empty string to accept connections from any interface. The port can be a number or a service name, such as 'http' or 'ftp'.
 - `socket.listen(backlog)`: This method enables the socket to accept incoming connections. The backlog argument specifies the maximum number of queued connections that can be waiting before the socket starts rejecting new ones.
 - `socket.accept()`: This method blocks until a new connection is established. It returns a pair of values: a new socket object that represents the connection, and the address of the client.
 - `socket.connect(address)`: This method connects the socket to a remote address and port. The address can be a hostname, an IP address, or a tuple of both. The port can be a number or a service name, such as 'http' or 'ftp'.
 - `socket.send(data)`: This method sends data to the connected socket. The data can be a string, a bytes object, or any object that supports the buffer protocol. The method returns the number of bytes sent, which may be less than the length of the data if the socket is non-blocking or the buffer is full.
 - `socket.recv(bufsize)`: This method receives data from the connected socket. The bufsize argument specifies the maximum number of bytes to read. The method returns a bytes object containing the data received, which may be empty if the socket is non-blocking or no data is available.
 - `socket.close()`: This method closes the socket and frees the resources associated with it. It also shuts down both ends of the con-

nection, preventing any further data transmission. The method does not return any value.

- **server.serve_forever()**: This method runs the server until it is stopped by calling **server.shutdown()** or by an exception. It handles incoming requests by creating a new thread for each one and calling the **server.handle_request()** method. The method does not return any value.

TCP/IP Client Sockets in Networking

- TCP/IP sockets are used to implement reliable, bidirectional, persistent, point-to-point, stream-based connections between hosts on the Internet.
- A socket is one endpoint of a two-way communication link between two programs running on the network.
- A socket is defined by the IP address of the machine and the port it uses.
- A socket address is the combination of an IP address and a port number, much like one end of a telephone connection is the combination of a phone number and a particular extension.
- To create a TCP/IP client socket in Java, you need to use the `java.net.Socket` class.
- The constructor for the `Socket` class has parameters that specify the address family, socket type, and protocol type that the socket uses to make connections.
- When connecting a client socket to a server socket, the client will use an `EndPoint` object to specify the network address of the server.
- To send and receive data over TCP, you need to use the `java.io.InputStream` and `java.io.OutputStream` classes that are obtained from the socket object.
- To close a TCP/IP client socket, you need to call the `close()` method on the socket object.

URL in Networking

- URL stands for Uniform Resource Locator and is a reference (an address) to a resource on the Internet .
- A URL consists of multiple parts, such as a protocol, a domain name, a path, and a query string, that tell a web browser how and where to retrieve a resource .
- For example, the URL `https://www.example.com/index.html?name=John` has the following components:
 - Protocol: `https`, which indicates the secure hypertext transfer protocol
 - Domain name: `www.example.com`, which identifies the server that hosts the resource
 - Path: `/index.html`, which specifies the location of the resource on the server
 - Query string: `?name=John`, which provides additional information

to the server

- A URL can point to different types of resources on the Internet, such as web pages, images, videos, database queries, command output, etc .
- A URL is important in networking because it allows users to access and share resources easily and uniformly across different platforms and applications .
- A URL is also a way of identifying and locating resources on the web, which is a large and distributed network of interconnected servers and clients .

URL Connection in Networking

- A URL connection is a way of establishing a communication link between a program and a resource identified by a URL (Uniform Resource Locator).
- A URL connection can be used to read or write data from or to the resource, such as a file, a web page, or a database query.
- A URL connection can also be used to get information about the resource, such as its type, size, last-modified date, or header fields.
- To create a URL connection, one needs to first create a URL object that represents the resource's location, and then call the `openConnection()` method on it. This returns a `URLConnection` object that can be cast to a specific subclass depending on the protocol of the URL, such as `URLConnection`, `JarURLConnection`, or `FileURLConnection`.
- A URL connection can be configured by setting various properties or options before connecting to the resource. For example, one can set the connection timeout, the request method, the request headers, or the proxy settings.
- A URL connection can be connected to the resource by calling the `connect()` method on it. This initiates the communication and may throw an `IOException` if an error occurs.
- A URL connection can be used to access the resource's content by getting an input stream or an output stream from it. For example, one can use the `getInputStream()` method to read data from the resource, or the `getOutputStream()` method to write data to the resource.
- A URL connection can be used to access the resource's metadata by getting various attributes or fields from it. For example, one can use the `getContentType()` method to get the MIME type of the resource, or the `getHeaderField()` method to get a specific header field value.
- A URL connection can be disconnected from the resource by calling the `disconnect()` method on it. This closes the connection and releases any resources associated with it.

TCP/IP Server Sockets in Networking

- A network socket is a software structure that serves as an endpoint for sending and receiving data across a network.

- A TCP/IP socket is a network socket that uses the Transmission Control Protocol (TCP) to establish a reliable and ordered data stream between two applications .
- A TCP/IP server socket is a TCP/IP socket that listens for incoming connections from TCP/IP client sockets on a specific port number .
- A TCP/IP server socket can accept multiple connections from different clients, but each connection is handled by a separate TCP/IP socket .
- To create a TCP/IP server socket, the following steps are required :
 - Create a Socket object with the address family, socket type, and protocol type that match the TCP/IP protocol.
 - Bind the Socket object to an IPEndPoint object that specifies the local IP address and port number to listen on.
 - Call the Listen method on the Socket object to start listening for incoming connections.
 - Call the Accept method on the Socket object to accept a connection from a client and return a new Socket object for that connection.
 - Use the new Socket object to send and receive data with the client.
 - Close the new Socket object when the communication is finished.
 - Repeat steps 4-6 for each incoming connection.
 - Close the original Socket object when the server is done listening.

Datagram in Networking

- A datagram is a basic transfer unit associated with a packet-switched network.
- Datagrams are data packets which contain adequate header information so that they can be individually routed by all intermediate network switching devices to the destination.
- Datagrams provide a connectionless communication service across a packet-switched network. This means that there is no need to establish or terminate a connection before or after sending data.
- In a datagram, data is frequently divided and transmitted from source to destination without a predefined route. The order of delivery to the receiver end is not guaranteed.
- Datagrams are typically structured in header and payload sections. The header contains information such as the source and destination addresses, the protocol type, the length, and the checksum. The payload contains the actual data to be transmitted.
- Datagrams are suitable for applications that require fast and efficient data transfer, such as video streaming, voice over IP, and online gaming. However, datagrams may also suffer from packet loss, duplication, delay, or corruption due to network congestion or errors.
- Datagrams are used by protocols such as the Internet Protocol (IP), the User Datagram Protocol (UDP), and the Internet Control Message Protocol (ICMP).

Unit 4 - Enterprise Java Bean

- Enterprise Java Beans (EJB) are server-side components that encapsulate the business logic of an application.
- EJBs are managed by an EJB container, which provides services such as security, transaction management, concurrency control, dependency injection, and remote access.
- EJBs can be of two types: session beans and message-driven beans.
- Session beans are stateful or stateless components that handle requests from clients. Stateful session beans maintain a conversational state with the client, while stateless session beans do not. Session beans can also be singleton, which means only one instance exists in the application.
- Message-driven beans are components that asynchronously process messages from a message queue or a topic. Message-driven beans do not have any state or conversational context with the client.
- EJBs can be accessed by local or remote clients using either Java Naming and Directory Interface (JNDI) or dependency injection. Local clients are in the same Java Virtual Machine (JVM) as the EJB, while remote clients are in a different JVM or a different machine.
- EJBs can use annotations or XML descriptors to specify their configuration and metadata, such as the type of the bean, the interfaces it implements, the transaction attributes, the security roles, and the lifecycle callbacks.
- EJBs can interact with other components and resources using the Java EE APIs, such as Java Persistence API (JPA), Java Transaction API (JTA), Java Message Service (JMS), JavaMail, Java Authentication and Authorization Service (JAAS), and Java Connector Architecture (JCA).
- EJBs can also use interceptors, which are classes that intercept the invocation of a method on an EJB and perform some pre-processing or post-processing logic. Interceptors can be used for logging, auditing, security, caching, or validation purposes.

Preparing a Class to be a JavaBeans

A JavaBeans is a reusable software component that can be manipulated visually in a builder tool. It follows certain coding conventions and design patterns that make it easy to use and integrate with other components. To prepare a class to be a JavaBeans, you need to follow these steps:

- Implement the `java.io.Serializable` interface. This allows the bean to be saved and restored in a persistent state .
- Provide a public no-argument constructor. This allows the bean to be instantiated by a builder tool or a container .
- Use private fields to store the properties of the bean. A property is a piece of data that describes the state or behavior of the bean .
- Provide public getter and setter methods for each property. A getter method returns the value of a property, and a setter method modifies

the value of a property. The names of the methods should follow the convention of `get<PropertyName>` and `set<PropertyName>` .

- Optionally, implement the `java.beans.PropertyChangeListener` interface or provide a `addPropertyChangeListener` and a `removePropertyChangeListener` method. This allows the bean to notify other components when a property changes .
- Optionally, implement the `java.beans.VetoableChangeListener` interface or provide a `addVetoableChangeListener` and a `removeVetoableChangeListener` method. This allows the bean to reject a property change if it is invalid or undesirable .
- Optionally, provide a `public static final String` field for each property name. This allows the bean to use constants instead of literals when firing property change events .
- Optionally, provide a custom `java.beans.BeanInfo` class or a `public static int` field named `ICON_COLOR_16x16`, `ICON_COLOR_32x32`, `ICON_MONO_16x16`, or `ICON_MONO_32x32`. This allows the bean to provide additional information or icons for a builder tool .

Here is an example of a simple JavaBeans class that represents a person:

```
import java.io.Serializable;
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

public class Person implements Serializable {

    // Constants for property names
    public static final String NAME_PROPERTY = "name";
    public static final String AGE_PROPERTY = "age";

    // Private fields for properties
    private String name;
    private int age;

    // Property change support
    private PropertyChangeSupport pcs;

    // Public no-argument constructor
    public Person() {
        this.name = "";
        this.age = 0;
        this.pcs = new PropertyChangeSupport(this);
    }

    // Getter and setter for name property
    public String getName() {
        return name;
    }
}
```

```

    }

    public void setName(String name) {
        String oldName = this.name;
        this.name = name;
        pcs.firePropertyChange(NAME_PROPERTY, oldName, name);
    }

    // Getter and setter for age property
    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        int oldAge = this.age;
        this.age = age;
        pcs.firePropertyChange(AGE_PROPERTY, oldAge, age);
    }

    // Methods for adding and removing property change listeners
    public void addPropertyChangeListener(PropertyChangeListener listener) {
        pcs.addPropertyChangeListener(listener);
    }

    public void removePropertyChangeListener(PropertyChangeListener listener) {
        pcs.removePropertyChangeListener(listener);
    }
}

```

Creating a JavaBeans

- A JavaBean is a reusable software component that follows certain design conventions and can be manipulated visually by a builder tool.
- To create a JavaBean, you need to follow these steps:
 - Define a public class with a no-argument constructor.
 - Provide public getter and setter methods for the properties of the bean that you want to expose.
 - Implement the `java.io.Serializable` interface to enable the bean to be saved and restored.
 - Optionally, implement the `java.beans.Customizer` interface to provide a custom GUI for editing the bean's properties.
 - Optionally, provide a `BeanInfo` class that describes the bean's properties, methods, events, and customizer.
 - Optionally, provide a manifest file that lists the bean classes and resources in a JAR file.
- To use a JavaBean, you need to import it into a builder tool, such as

NetBeans or Eclipse, and drag and drop it onto a container, such as a JFrame or a JPanel. You can then edit the bean's properties, methods, and events using the tool's property sheet, code editor, and event handler. You can also run and test the bean using the tool's debugger and tester.

JavaBeans Properties

- A JavaBean property is a named attribute that can be accessed by the user of the object.
- The property can be of any Java data type, including the classes that you define.
- A property can be read-only, write-only, or read-write, depending on the presence of getter and setter methods.
- A getter method is a public method that returns the value of the property. It has the form `public <type> get<PropertyName>()`.
- A setter method is a public method that sets the value of the property. It has the form `public void set<PropertyName>(<type> value)`.
- A property can also be bound or constrained, meaning that it can notify other objects when its value changes, or it can allow other objects to veto its value change.
- A bound property has a pair of methods `public void addPropertyChangeListener(PropertyChangeListener listener)` and `public void removePropertyChangeListener(PropertyChangeListener listener)` that allow other objects to register or unregister as listeners for property change events.
- A constrained property has a pair of methods `public void addVetoableChangeListener(VetoableChangeListener listener)` and `public void removeVetoableChangeListener(VetoableChangeListener listener)` that allow other objects to register or unregister as listeners for vetoable change events.
- A vetoable change event is a special kind of property change event that can be rejected by any of the listeners, causing the property value to revert to its previous value.
- A property can also be indexed, meaning that it can have multiple values, accessed by an integer index. An indexed property has a pair of getter and setter methods that take an additional int parameter to specify the index. For example, `public <type> get<PropertyName>(int index)` and `public void set<PropertyName>(int index, <type> value)`.
- A JavaBean can have any number of properties, of any type and access mode. The properties of a JavaBean can be discovered by other tools or components using the JavaBeans Introspector class, which uses reflection to analyze the methods of the class and infer the properties.

Types of beans in Enterprise Java Bean

Enterprise Java Bean (EJB) is a Java component that can be used to create Java applications that run on a server. EJBs can contain business logic, data access, and messaging functionality. There are three types of EJBs, each with

different characteristics and use cases :

- **Session beans:** Session beans encapsulate business logic that can be invoked by a client, either locally or remotely. Session beans do not have a persistent state, meaning they do not store data across invocations. There are three subtypes of session beans:
 - **Stateless session beans:** Stateless session beans do not maintain any conversational state with the client. They are typically used for simple and short-lived operations, such as validating user credentials or performing calculations. Stateless session beans can be pooled and reused by different clients, improving performance and scalability.
 - **Stateful session beans:** Stateful session beans maintain a conversational state with the client, meaning they can store data and information related to a specific client session. They are typically used for complex and long-lived operations, such as shopping carts or online banking transactions. Stateful session beans are not pooled and are tied to a single client, requiring more resources and memory.
 - **Singleton session beans:** Singleton session beans are instantiated only once per application and are shared by all clients. They are typically used for global and application-wide operations, such as caching, logging, or configuration. Singleton session beans can be accessed concurrently by multiple threads, requiring synchronization and locking mechanisms to ensure data consistency and integrity.
- **Message-driven beans:** Message-driven beans are a type of session bean that process messages asynchronously from a message queue or topic. Message-driven beans are typically used for event-driven and decoupled operations, such as sending notifications, processing orders, or updating inventory. Message-driven beans do not have a conversational state with the client or the message provider, and can be pooled and reused by different messages, improving performance and scalability.
- **Entity beans:** Entity beans are deprecated since EJB 3.0 and are replaced by Java Persistence API (JPA) entities. Entity beans used to represent persistent data stored in a database, and provide methods for creating, retrieving, updating, and deleting data. Entity beans had two subtypes:
 - **Container-managed persistence (CMP) entity beans:** CMP entity beans delegated the data access and mapping logic to the EJB container, which automatically generated the SQL queries and statements based on the entity bean definition and configuration. CMP entity beans were easier to develop and maintain, but had less flexibility and control over the data access and mapping logic.
 - **Bean-managed persistence (BMP) entity beans:** BMP entity beans implemented the data access and mapping logic manually, using JDBC or other APIs to interact with the database. BMP entity beans had more flexibility and control over the data access and mapping logic, but required more coding and testing effort.

Stateful Session bean in Enterprise Java Bean

- A stateful session bean is a type of enterprise bean that maintains a conversational state with a client.
- A stateful session bean can store the state of a client across multiple method invocations, such as the items in a shopping cart or the progress of a workflow.
- A stateful session bean is identified by a unique identifier that is assigned by the container when the bean is created. The identifier is used to locate the bean instance when the client invokes a method on it.
- A stateful session bean can be passivated by the container to conserve memory. Passivation means that the bean instance is serialized and written to a secondary storage, such as a file or a database. The bean instance is reactivated when the client invokes a method on it again.
- A stateful session bean can be removed by the container when it is not needed anymore, such as when the client invokes a method annotated with `@Remove` or when the bean instance exceeds a timeout period. The bean instance can also be removed by the client explicitly by invoking a method on the `EJBHome` or `EJBLocalHome` interface.
- A stateful session bean can implement the `SessionSynchronization` interface to receive notifications from the container before and after the bean instance is passivated or activated. The bean can use these notifications to perform any necessary cleanup or initialization tasks.
- A stateful session bean can implement the `javax.ejb.SessionBean` interface to receive lifecycle callbacks from the container, such as `ejbCreate`, `ejbRemove`, `ejbActivate`, and `ejbPassivate`. The bean can use these callbacks to perform any custom logic or resource management.
- A stateful session bean can be accessed by local or remote clients through a business interface or a home interface. A business interface defines the business methods of the bean, while a home interface defines the methods for creating, finding, and removing the bean instances. A stateful session bean can also expose a web service endpoint by annotating the bean class or the business interface with `@WebService`.

Stateless Session bean in Enterprise Java Bean

- A stateless session bean is a type of enterprise bean that does not maintain any conversational state with the client.
- A stateless session bean is typically used to implement business logic or service methods that are independent of any specific client.
- A stateless session bean can be invoked by multiple clients concurrently, and the container can pool and reuse the bean instances to improve performance and scalability.
- A stateless session bean has the following characteristics:
 - It does not have any instance variables that store client-specific data or conversational state.

- It does not implement the `SessionSynchronization` interface, which is used to manage transactions and concurrency.
- It can implement the `@PrePassivate` and `@PostActivate` lifecycle callbacks, which are invoked by the container before and after the bean instance is passivated (removed from the pool) or activated (restored to the pool).
- It can implement the `@Remove` method, which is invoked by the client or the container to indicate that the bean instance is no longer needed and can be removed from the pool.
- It can implement the `@PostConstruct` and `@PreDestroy` lifecycle callbacks, which are invoked by the container when the bean instance is created or destroyed.
- It can implement the `@AroundInvoke` method, which is used to intercept and modify the invocation of the business methods.
- It can implement the `@Asynchronous` annotation, which indicates that the business methods can be invoked asynchronously by the client or the container.
- It can implement the `@Timeout` annotation, which indicates that the bean instance can receive timer callbacks from the container.
- It can implement the `@Schedule` or `@Schedules` annotation, which indicates that the bean instance can receive scheduled timer callbacks from the container.
- It can implement the `@WebService` or `@WebMethod` annotation, which indicates that the bean instance can expose its business methods as web services.
- It can implement the `@RolesAllowed`, `@PermitAll`, `@DenyAll`, or `@DeclareRoles` annotation, which indicates the security roles and permissions for accessing the bean instance and its business methods.
- It can implement the `@Resource`, `@EJB`, `@PersistenceContext`, `@PersistenceUnit`, `@WebServiceRef`, or `@Inject` annotation, which indicates the resources or dependencies that the bean instance can access or inject from the container.

Entity bean in Enterprise Java Bean

- An entity bean is a type of Enterprise Java Bean (EJB), which is a server-side Java EE component that encapsulates business logic and data access.
- An entity bean represents persistent data maintained in a database. It can map to one or more tables in the database and provide an object-oriented view of the data.
- An entity bean can manage its own persistence (Bean managed persistence or BMP) or can delegate this function to its EJB Container (Container managed persistence or CMP).
- An entity bean is identified by a primary key, which is a unique value that distinguishes one entity bean instance from another.
- An entity bean can have local or remote interfaces, which define the meth-

ods that clients can invoke on the entity bean. A local interface is used when the client and the entity bean are in the same JVM, while a remote interface is used when they are in different JVMs.

- An entity bean can also have a home interface, which defines the methods for creating, finding, and removing entity bean instances. A home interface can be local or remote as well.
- An entity bean can be either session-aware or session-independent. A session-aware entity bean is associated with a specific client session and can maintain state across method invocations. A session-independent entity bean is not tied to any client session and does not maintain state.
- An entity bean can be either reentrant or non-reentrant. A reentrant entity bean can be accessed concurrently by multiple threads, while a non-reentrant entity bean can only be accessed by one thread at a time.
- An entity bean can implement various callback methods to perform custom operations during its lifecycle, such as `ejbCreate`, `ejbLoad`, `ejbStore`, `ejbRemove`, etc.
- An entity bean can participate in transactions, security, and concurrency management provided by the EJB Container. It can also use other EJB components, such as session beans and message-driven beans, to implement business logic and communication.

Java Database Connectivity (JDBC)

- JDBC is an API (Application Programming Interface) that allows Java programs to interact with databases and data sources.
- JDBC provides a standard set of methods and classes for connecting to different types of databases, executing SQL statements, retrieving results, and handling errors.
- JDBC supports various types of data sources, such as relational databases, spreadsheets, flat files, and web services.
- JDBC consists of four main components: drivers, connections, statements, and result sets.
 - Drivers are software modules that implement the JDBC interface and communicate with a specific type of database or data source. Drivers are loaded dynamically at runtime and registered with the JDBC driver manager.
 - Connections are objects that represent a logical link between a Java program and a database or data source. Connections are obtained from a driver or a data source object, and can be used to create statements and transactions.
 - Statements are objects that represent SQL commands or queries that can be executed on a database or data source. Statements can be of three types: simple, prepared, and callable. Simple statements are used for executing static SQL statements. Prepared statements are used for executing parameterized SQL statements. Callable statements are used for executing stored procedures or functions.

- Result sets are objects that hold the data returned by a statement execution. Result sets can be of two types: forward-only and scrollable. Forward-only result sets can only be traversed in one direction, from the first row to the last row. Scrollable result sets can be traversed in both directions, and can also be positioned at a specific row or column.
- JDBC also provides some optional features, such as batch updates, metadata, row sets, data types, and transactions. Batch updates allow multiple SQL statements to be executed as a single unit. Metadata provides information about the database, the tables, the columns, and the result sets. Row sets are special result sets that can be disconnected from the database and manipulated in memory. Data types are classes that map Java types to SQL types and vice versa. Transactions are units of work that ensure the consistency and integrity of the data.

Merging Data from Multiple Tables in JDBC

- JDBC stands for Java Database Connectivity, which is an API that allows Java programs to interact with various types of databases.
- One of the common tasks in JDBC is to merge data from multiple tables, which can be done using SQL queries that join the tables based on some common attributes or conditions.
- There are different types of joins in SQL, such as inner join, left join, right join, full join, cross join, etc. Each type of join has a different syntax and result set, depending on how the tables are related and what data is required.
- To merge data from multiple tables in JDBC, the following steps are needed:
 - Create a Connection object that represents the connection to the database.
 - Create a Statement object that can execute SQL queries on the database.
 - Write a SQL query that joins the tables using the appropriate join type and conditions.
 - Execute the query using the `executeQuery()` method of the Statement object, which returns a `ResultSet` object that contains the merged data.
 - Iterate over the `ResultSet` object using the `next()` method and access the data using the `getXXX()` methods, where XXX is the data type of the column.
 - Close the `ResultSet`, `Statement`, and `Connection` objects using the `close()` method to release the resources.
- Here is an example of merging data from two tables in JDBC using an inner join:

// Assume that the tables are as follows:

```

// Employee (emp_id, name, dept_id, salary)
// Department (dept_id, dept_name, location)

// Import the required packages
import java.sql.*;

public class MergeData {

    public static void main(String[] args) {
        // Declare the JDBC objects
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;

        try {
            // Load the JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Establish the connection to the database
            conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb", "root", "pass");

            // Create the statement object
            stmt = conn.createStatement();

            // Write the SQL query that joins the tables
            String sql = "SELECT e.name, e.salary, d.dept_name, d.location FROM Employee e INNER JOIN Department d ON e.dept_id = d.dept_id";

            // Execute the query and get the result set
            rs = stmt.executeQuery(sql);

            // Iterate over the result set and print the data
            while (rs.next()) {
                // Get the data from the columns
                String name = rs.getString("name");
                double salary = rs.getDouble("salary");
                String dept_name = rs.getString("dept_name");
                String location = rs.getString("location");

                // Print the data
                System.out.println(name + "\t" + salary + "\t" + dept_name + "\t" + location);
            }
        } catch (Exception e) {
            // Handle any errors
            e.printStackTrace();
        } finally {
            // Close the JDBC objects

```

```

    try {
        if (rs != null) {
            rs.close();
        }
        if (stmt != null) {
            stmt.close();
        }
        if (conn != null) {
            conn.close();
        }
    } catch (SQLException se) {
        se.printStackTrace();
    }
}
}
}

```

Joining in JDBC

- Joining in JDBC is a technique to combine data from two or more tables based on a common column or condition.
- Joining in JDBC can be performed by using the SQL JOIN clause in the query statement.
- There are different types of joins in SQL, such as inner join, outer join, cross join, self join, etc.
- To perform a join operation in JDBC, the following steps are required:
 - Import the JDBC packages, such as `java.sql.*` and `javax.sql.*`.
 - Register and load the JDBC driver, such as `Class.forName("com.mysql.jdbc.Driver")`.
 - Set a connection to the database, such as `Connection con = DriverManager.getConnection(url, user, password)`.
 - Create a statement object to execute the query, such as `Statement stmt = con.createStatement()`.
 - Write the query statement with the JOIN clause, such as `String query = "SELECT * FROM Product INNER JOIN Orders ON Product.ItemID = Orders.ItemID"`.
 - Execute the query and obtain the result set, such as `ResultSet rs = stmt.executeQuery(query)`.
 - Work on the result set, such as iterating over the rows and columns, or displaying the data.
 - Close the statement, result set, and connection objects, such as `stmt.close()`, `rs.close()`, and `con.close()`.
- To perform a join operation with more than two tables, the JOIN clause can be repeated for each table, such as `SELECT * FROM Product INNER JOIN Orders ON Product.ItemID = Orders.ItemID LEFT OUTER JOIN SupplierProduct ON Orders.ItemID = SupplierProduct.ItemID`.
- To perform a join operation with RowSet objects, the JoinRowSet interface

can be used, which extends the `WebRowSet` interface. A `JoinRowSet` object can add any `RowSet` object that can be part of a SQL JOIN, such as `JdbcRowSet`, `CachedRowSet`, etc. Each `RowSet` object added to a `JoinRowSet` object must have a match column, which is the column on which the JOIN is based. The `JoinRowSet` object can then perform the join operation internally and provide the result set.

Manipulating in JDBC

JDBC stands for Java Database Connectivity, which is an API that allows Java programs to communicate with databases and manipulate their data. JDBC supports various types of databases, but it is mainly used with relational databases.

To manipulate data in a database using JDBC, you need to follow these steps:

- Load the JDBC driver class that corresponds to the database you want to use. For example, for Oracle database, you need to load the `oracle.jdbc.driver.OracleDriver` class. You can use the `Class.forName()` method to load the driver class.
- Establish a connection to the database using the `DriverManager.getConnection()` method. You need to provide the URL, username and password of the database as parameters. For example, for Oracle database, the URL format is `jdbc:oracle:thin:@host:port:SID`.
- Create a `Statement` object using the `Connection.createStatement()` method. A `Statement` object allows you to execute SQL statements against the database.
- Execute the SQL statement using the `Statement.execute()`, `Statement.executeQuery()` or `Statement.executeUpdate()` methods. The `execute()` method returns a boolean value indicating whether the statement returns a `ResultSet` object or not. The `executeQuery()` method returns a `ResultSet` object that contains the data retrieved from the database. The `executeUpdate()` method returns an `int` value indicating the number of rows affected by the statement.
- Process the `ResultSet` object if the statement returns one. A `ResultSet` object represents a table of data that can be accessed by column name or index. You can use the `ResultSet.next()` method to move the cursor to the next row and the `ResultSet.getXXX()` methods to retrieve the values of the columns. For example, `ResultSet.getString(1)` returns the value of the first column as a `String`.
- Close the `ResultSet`, `Statement` and `Connection` objects using the `close()` methods. This releases the resources and prevents memory leaks.

Here is an example of manipulating data in an Oracle database using JDBC:

```
// Load the Oracle JDBC driver
Class.forName("oracle.jdbc.driver.OracleDriver");

// Connect to the database
```

```

Connection conn = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:XE", "scott", "tiger");

// Create a statement
Statement stmt = conn.createStatement();

// Execute a query
ResultSet rs = stmt.executeQuery("SELECT * FROM emp");

// Process the result set
while (rs.next()) {
    // Print the employee name and salary
    System.out.println(rs.getString("ename") + " " + rs.getInt("sal"));
}

// Close the result set, statement and connection
rs.close();
stmt.close();
conn.close();

```

Databases with JDBC in JDBC

- JDBC stands for Java Database Connectivity, which is an API for connecting and executing queries on databases from Java applications.
- JDBC provides a common interface for different database systems, such as MySQL, Oracle, PostgreSQL, etc. by using drivers that implement the JDBC interfaces.
- JDBC uses a URL format to specify the connection parameters for a database, such as the protocol, host, port, database name, and properties.
- JDBC supports various types of connections, such as direct, pooled, or distributed, depending on the application needs and the database system.
- JDBC allows the application to perform CRUD (Create, Read, Update, Delete) operations on the database using SQL statements, which can be executed either as plain text or as prepared statements.
- JDBC also provides features such as transactions, batch updates, metadata, result sets, and stored procedures to handle complex database operations.

Prepared Statements in JDBC

- Prepared statements are a special type of statements that are derived from the Statement interface.
- Prepared statements are used to execute parameterized queries against the database .
- Parameterized queries are SQL queries that contain placeholders (represented by ? symbol) for the values that will be supplied at runtime .

- Prepared statements are precompiled by the database and can be executed multiple times with different values .
- Prepared statements have several advantages over regular statements, such as:
 - Improved performance: The database only needs to compile the query once and can reuse the same execution plan for subsequent executions .
 - Enhanced security: The values are passed as parameters and not as part of the SQL query, which prevents SQL injection attacks .
 - Increased readability: The SQL query is more concise and clear, as it does not contain the actual values .
- To use prepared statements, the following steps are required:
 - Create a PreparedStatement object by calling the prepareStatement() method of the Connection object and passing the SQL query with placeholders as an argument .
 - Set the values for the placeholders by calling the appropriate setter methods of the PreparedStatement object, such as setInt(), setString(), setDouble(), etc. The first argument of these methods specifies the index of the placeholder (starting from 1), and the second argument specifies the value .
 - Execute the query by calling the executeQuery() or executeUpdate() method of the PreparedStatement object, depending on the type of the query (select or update/insert/delete) .
 - Process the result set (if any) by using the ResultSet object returned by the executeQuery() method .
 - Close the PreparedStatement and ResultSet objects by calling their close() methods .
- An example of using prepared statements in JDBC is given below:

```
// Assume conn is an already created Connection object
// Create a SQL query with two placeholders
String sql = "SELECT * FROM employees WHERE name = ? AND salary > ?";
// Create a PreparedStatement object
PreparedStatement pstmt = conn.prepareStatement(sql);
// Set the values for the placeholders
pstmt.setString(1, "John"); // name
pstmt.setDouble(2, 5000.0); // salary
// Execute the query
ResultSet rs = pstmt.executeQuery();
// Process the result set
while (rs.next()) {
    // Retrieve the data from each row
    int id = rs.getInt("id");
    String name = rs.getString("name");
    double salary = rs.getDouble("salary");
    // Print the data
}
```

```

        System.out.println(id + " " + name + " " + salary);
    }
    // Close the resources
    rs.close();
    pstmt.close();

```

Transaction Processing in JDBC

- Transaction processing is a mandatory requirement of all applications that must guarantee consistency of their persistent data .
- Transactions are atomic, consistent, isolated, and durable (ACID) modules of execution .
- Atomicity means either all successful or none.
- Consistency ensures bringing the database from one consistent state to another consistent state.
- Isolation ensures that transaction is isolated from other transaction.
- Durability ensures that the changes made by the transaction are permanent.
- In JDBC, every SQL query will be considered as a transaction.
- When we create a Database connection in JDBC, it will run in auto-commit mode (auto-commit value is TRUE) .
- Auto-commit mode means that after the execution of the SQL statement, it will be committed automatically .
- To disable auto-commit mode, we can use the `setAutoCommit(false)` method of the Connection interface .
- To commit a transaction, we can use the `commit()` method of the Connection interface .
- To rollback a transaction, we can use the `rollback()` method of the Connection interface .
- To set and rollback to savepoints, we can use the `setSavepoint()` and `rollback(Savepoint)` methods of the Connection interface.
- Transaction processing can either be performed locally or distributed .
- Local transactions are typically managed explicitly by the application or automatically by the Java Platform, Enterprise Edition (Java EE) application server.
- Distributed transactions are managed by a transaction manager that coordinates the work of multiple resource managers.

Stored Procedures in JDBC

- Stored procedures are subroutines, segments of SQL statements that are stored in the SQL catalog of a database.
- Stored procedures can be accessed by applications that can access relational databases, such as Java, Python, PHP, etc.
- Stored procedures can improve the performance and security of database applications by reducing the network traffic and enforcing access controls.

- Stored procedures can also return output parameters or result sets to the calling applications.
- JDBC provides a standard SQL escape syntax for calling stored procedures in all RDBMSs.
- To call a stored procedure using JDBC, you need to follow these steps:
 - Register the driver class using the `registerDriver()` method of the `DriverManager` class.
 - Establish a connection to the database using the `getConnection()` method of the `DriverManager` class.
 - Create a `CallableStatement` object using the `prepareCall()` method of the `Connection` object. The `prepareCall()` method takes a string argument that specifies the SQL escape syntax for calling the stored procedure.
 - If the stored procedure has input parameters, set their values using the `setXXX()` methods of the `CallableStatement` object, where `XXX` is the data type of the parameter.
 - If the stored procedure has output parameters, register their data types using the `registerOutParameter()` method of the `CallableStatement` object.
 - Execute the stored procedure using the `execute()` or `executeQuery()` method of the `CallableStatement` object.
 - If the stored procedure returns output parameters, retrieve their values using the `getXXX()` methods of the `CallableStatement` object, where `XXX` is the data type of the parameter.
 - If the stored procedure returns a result set, process it using the `ResultSet` object returned by the `executeQuery()` method of the `CallableStatement` object.
 - Close the `CallableStatement` and `Connection` objects using the `close()` method.

Unit 5 - Servlets

- Servlets are Java classes that run on a web server and handle requests from web clients (such as browsers).
- Servlets can generate dynamic web pages, interact with databases, process forms, manage sessions, and perform other web-related tasks.
- Servlets are managed by a servlet container, which is a component of a web server that provides the runtime environment and services for servlets.
- Servlets follow a request-response model, where the servlet container receives an HTTP request from a client, invokes the appropriate servlet, and sends back an HTTP response to the client.
- Servlets implement the `javax.servlet.Servlet` interface, which defines the lifecycle methods and the service method for handling requests.
- Servlets can be configured using annotations or deployment descriptors (XML files).
- Servlets can access various objects and information from the servlet con-

tainer, such as the servlet context, the servlet config, the request, the response, the session, and the listeners.

- Servlets can use filters to intercept and modify requests and responses before and after they reach the servlet.
- Servlets can use listeners to monitor and react to events in the servlet context, the session, the request, and the attributes.
- Servlets can use cookies and URL rewriting to track the state of a client across multiple requests.
- Servlets can use JavaServer Pages (JSP) to simplify the creation of dynamic web pages using HTML, XML, and Java code.

Servlet Overview and Architecture in Servlets

- A servlet is a Java class that implements the `javax.servlet.Servlet` interface and runs on a web server to handle HTTP requests and generate dynamic web content.
- A servlet can receive parameters from a web browser, process them, access databases or other resources, and generate HTML, XML, JSON, or other types of responses.
- A servlet is managed by a servlet container, which is a component of a web server that provides the environment and services for servlets to run.
- A servlet container is responsible for loading and initializing servlets, invoking their methods, and managing their life cycle.
- A servlet container also handles the communication between the servlet and the web server, as well as the security, concurrency, and performance aspects of servlet execution.
- A servlet container can support multiple servlets, each mapped to a specific URL pattern or request type.
- A servlet can also communicate with other servlets or components using the servlet context, which is an object that represents the shared information and functionality of a web application.
- A servlet can also use the servlet configuration, which is an object that contains the initialization parameters and other information specific to a servlet instance.
- A servlet can also use the servlet request and servlet response objects, which encapsulate the information and functionality related to an HTTP request and response, such as headers, parameters, cookies, sessions, streams, etc.
- A servlet can also use filters, which are components that can intercept and modify the requests and responses before or after they reach the servlet.
- A servlet can also use listeners, which are components that can perform actions based on the events that occur in the servlet context, such as creation, destruction, attribute changes, etc.

Interface Servlet and the Servlet Life Cycle in Servlets

- A servlet is a Java class that runs on a web server and handles HTTP requests and responses.
- All servlets must implement the `javax.servlet.Servlet` interface, which defines the methods to initialize a servlet, to service requests, and to destroy a servlet from the server .
- These methods are known as **life cycle methods**, and are called by the web container (the application that manages the servlets) at different stages of the servlet's existence .
- The life cycle of a servlet consists of the following stages :
 - **Servlet is loaded:** The web container loads the servlet class into memory when it is requested for the first time, or when the server starts up.
 - **Servlet is initialized:** The web container invokes the `init()` method of the servlet, which corresponds to the initialization phase of the servlet life cycle. The `init()` method receives a `ServletConfig` object that contains the initialization parameters and the servlet context. The `init()` method can be used to perform any one-time tasks, such as creating database connections, initializing resources, etc. The `init()` method is called only once during the servlet's lifetime .
 - **Servlet is ready to service:** After the `init()` method completes, the servlet is ready to handle client requests. The web container creates a separate thread for each request and passes the request and response objects to the servlet.
 - **Servlet is servicing:** The web container calls the `service()` method of the servlet for each request. The `service()` method determines the type of the request (GET, POST, PUT, DELETE, etc.) and dispatches it to the appropriate handler method, such as `doGet()`, `doPost()`, etc. The handler methods can access the request parameters, headers, cookies, etc. and generate the response content, status, headers, cookies, etc. The `service()` method can be overridden by the servlet, but it is not recommended .
 - **Servlet is not ready to service:** The servlet can become unavailable to service requests for various reasons, such as being temporarily disabled, being under maintenance, or being overloaded. The web container can call the `isUnavailable()` method of the servlet to check its availability status. The servlet can also notify the web container of its unavailability by throwing an `UnavailableException`.
 - **Servlet is destroyed:** The web container invokes the `destroy()` method of the servlet, which corresponds to the destruction phase of the servlet life cycle. The `destroy()` method can be used to perform any final tasks, such as releasing resources, closing connections, etc.

The `destroy()` method is called only once at the end of the servlet's lifetime, or when the web container shuts down .

- The following diagram illustrates the servlet life cycle:

Servlet Life Cycle

- The servlet life cycle methods are central to the functioning of a servlet, and they should be implemented carefully and correctly. The servlet interface provides default implementations for these methods, but they are usually overridden by the servlet subclasses, such as `HttpServlet`, `GenericServlet`, etc. The servlet subclasses provide more convenience and flexibility for handling different types of requests and responses .

Handling HTTP get Requests in Servlets

- HTTP get requests are used to retrieve information from a server based on the parameters specified in the URL.
- To handle HTTP get requests in a servlet, you need to extend the `HttpServlet` class and override the `doGet()` method.
- The `doGet()` method takes two parameters: `HttpServletRequest` and `HttpServletResponse`, which represent the request and response objects respectively.
- The `HttpServletRequest` object provides methods to access the request information, such as the URL, the query string, the headers, the cookies, etc.
- The `HttpServletResponse` object provides methods to set the response information, such as the status code, the headers, the content type, the body, etc.
- The `doGet()` method can also handle HTTP head requests, which are similar to get requests but only return the headers and not the body of the response.
- The `doGet()` method can throw `ServletException` or `IOException`, which indicate that an error occurred while processing the request or writing the response.
- The `doGet()` method can also call other methods of the `HttpServlet` class, such as `getServletContext()`, `getServletConfig()`, `getInitParameter()`, etc., to access the servlet context, configuration, and initialization parameters.
- The `doGet()` method can also forward the request to another servlet or JSP page, or include the output of another servlet or JSP page, using the `RequestDispatcher` object obtained from the request or the servlet context.
- The `doGet()` method can also use the `PrintWriter` object obtained from the response to write the response body in text format, or the `ServletOutputStream` object obtained from the response to write the response body in binary format.

Handling HTTP post Requests in Servlets

- HTTP post requests are used to send data to the server in the body of the request, such as form data, files, etc.
- To handle HTTP post requests in a servlet, you need to extend the `HttpServlet` class and override the `doPost` method.
- The `doPost` method takes two parameters: a `HttpServletRequest` object and a `HttpServletResponse` object.
- The `HttpServletRequest` object provides methods to access the request data, such as `getParameter`, `getInputStream`, `getHeader`, etc.
- The `HttpServletResponse` object provides methods to send the response data, such as `setContentType`, `getOutputStream`, `addHeader`, etc.
- You can use the `request.getParameter` method to get the values of the form fields submitted by the client.
- You can use the `response.setContentType` method to set the MIME type of the response, such as `text/html`, `application/json`, etc.
- You can use the `response.getOutputStream` method to get a `ServletOutputStream` object that allows you to write the response data in bytes.
- You can use the `response.getWriter` method to get a `PrintWriter` object that allows you to write the response data in characters.
- You can use the `response.addHeader` method to add HTTP headers to the response, such as `Content-Disposition`, `Cache-Control`, etc.
- You can use the `response.sendRedirect` method to redirect the client to another URL.
- You can use the `response.sendError` method to send an error code and a message to the client.
- You can use the `request.getRequestDispatcher` method to get a `RequestDispatcher` object that allows you to forward the request to another servlet or a JSP page.
- You can use the `request.setAttribute` and `request.getAttribute` methods to store and retrieve data in the request scope.
- You can use the `request.getSession` method to get a `HttpSession` object that allows you to store and retrieve data in the session scope.
- You can use the `request.getCookies` and `response.addCookie` methods to get and set cookies in the request and response.

Redirecting Requests to Other Resources in Servlets

- Redirecting requests to other resources in servlets means sending the response to a different resource than the one that received the request.
- A resource can be another servlet, a JSP page, an HTML file, or any other web component.
- Redirecting requests can be useful for implementing features like authentication, error handling, logging, etc.
- There are two ways to redirect requests to other resources in servlets: using

the `sendRedirect()` method of the `HttpServletResponse` interface, or using the `forward()` method of the `RequestDispatcher` interface.

Using the `sendRedirect()` Method

- The `sendRedirect()` method of the `HttpServletResponse` interface can be used to redirect the response to another resource by specifying its URL as a parameter.
- The syntax of the `sendRedirect()` method is:

```
public void sendRedirect(String url) throws IOException
```

- The `sendRedirect()` method works on the client side, meaning that it instructs the browser to make a new request to the specified URL.
- The URL can be relative or absolute, and it can point to a resource inside or outside the server.
- The `sendRedirect()` method also sends a temporary redirect response to the client with a 3XX status code, indicating that the resource has moved to a different location.
- The advantages of using the `sendRedirect()` method are:
 - It can redirect the response to any resource, even on a different server or domain.
 - It can prevent the user from resubmitting the same request by refreshing the browser, as the original request is lost.
- The disadvantages of using the `sendRedirect()` method are:
 - It is slower than the `forward()` method, as it involves two round trips between the client and the server.
 - It cannot share the request and response objects with the new resource, as they are discarded after the first response.
 - It cannot hide the URL of the new resource from the user, as it is visible in the browser's address bar.

Using the `forward()` Method

- The `forward()` method of the `RequestDispatcher` interface can be used to forward the request and response objects to another resource within the same web application.
- The syntax of the `forward()` method is:

```
public void forward(ServletRequest request, ServletResponse response) throws ServletException
```

- The `forward()` method works on the server side, meaning that it transfers the control to the new resource without involving the browser.
- The URL of the new resource is not visible to the user, as the browser's address bar remains unchanged.

- The `forward()` method also preserves the request and response objects, allowing the new resource to access the data and parameters from the original request.
- The advantages of using the `forward()` method are:
 - It is faster than the `sendRedirect()` method, as it avoids the extra network traffic between the client and the server.
 - It can share the request and response objects with the new resource, enabling data transfer and communication between the web components.
 - It can hide the URL of the new resource from the user, enhancing the security and privacy of the web application.
- The disadvantages of using the `forward()` method are:
 - It can only forward the request to a resource within the same web application, not to a different server or domain.
 - It can cause problems if the response has already been committed before the `forward()` method is called, as it cannot modify the response headers or content.

Session Tracking in Servlets

- Session tracking is a mechanism that allows a web server to maintain the state of a user across multiple requests.
- A session is a collection of data associated with a user and stored on the server side.
- Session tracking is useful for implementing features such as shopping carts, authentication, personalization, etc.
- Servlets provide four ways of session tracking: cookies, URL rewriting, hidden form fields, and `HttpSession` objects.
- Cookies are small pieces of data that are sent by the server to the client and stored in the client's browser. They can be used to store session IDs or other information that can be retrieved by the server in subsequent requests.
- URL rewriting is a technique that appends the session ID or other information to the URL of each request. The server can extract the session ID from the URL and use it to identify the user.
- Hidden form fields are input elements in HTML forms that are not visible to the user. They can be used to store session IDs or other information that can be submitted to the server along with the form data.
- `HttpSession` objects are Java objects that are created by the server and associated with a user. They can store any type of data as attributes and have methods to manipulate them. They can be accessed by the servlets using the `HttpServletRequest.getSession()` method.

Cookies in Servlets

- Cookies are small pieces of information that are stored in the client's browser and sent to the server with every request .
- Cookies are used for state management and session tracking, as the server treats every client request as a new one by default.
- Cookies are created using the `Cookie` class in the `javax.servlet.http` package .
- To create a cookie, we need to pass the name and value of the cookie to the constructor of the `Cookie` class, for example:

```
Cookie cookie = new Cookie("username", "John");
```

- To send a cookie to the client, we need to add it to the response object using the `addCookie()` method, for example:

```
response.addCookie(cookie);
```

- To retrieve a cookie from the client, we need to get the array of cookies from the request object using the `getCookies()` method, and then loop through the array to find the cookie by name, for example:

```
Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie c : cookies) {
        if (c.getName().equals("username")) {
            String username = c.getValue();
            // do something with username
        }
    }
}
```

- Cookies have optional attributes such as `comment`, `path`, `domain`, `maxAge`, and `version` that can be set or get using the corresponding methods of the `Cookie` class.
- For example, to set the expiration date of a cookie, we can use the `setMaxAge()` method, which takes the number of seconds as a parameter, for example:

```
cookie.setMaxAge(60 * 60 * 24); // expires in one day
```

- Cookies are not very secure, as they can be easily modified or deleted by the client, or intercepted by a third party. Therefore, sensitive information should not be stored in cookies.

Session Tracking with Http Session in Servlets

- Session tracking is a mechanism to maintain the state of a user across multiple requests.
- Http session is one of the ways to implement session tracking in servlets.

- Http session is an object that represents a unique session between a client and a server.
- Http session is created by the server and assigned to the client when the client makes the first request to the server.
- Http session is identified by a session ID, which is a unique string generated by the server and sent to the client as a cookie or a URL parameter.
- Http session can store attributes, which are key-value pairs of data that can be accessed by the servlets within the same session.
- Http session has a timeout period, which is the maximum time interval between two consecutive requests from the same client. If the timeout period expires, the session is invalidated by the server and the attributes are discarded.
- Http session can be manipulated by the servlets using the methods of the `javax.servlet.http.HttpSession` interface, such as:
 - `getId()`: returns the session ID of the session.
 - `getCreationTime()`: returns the time when the session was created.
 - `getLastAccessedTime()`: returns the time when the session was last accessed by the client.
 - `getMaxInactiveInterval()`: returns the timeout period of the session in seconds.
 - `setMaxInactiveInterval(int interval)`: sets the timeout period of the session in seconds.
 - `getAttribute(String name)`: returns the value of the attribute with the given name, or null if not found.
 - `setAttribute(String name, Object value)`: sets the value of the attribute with the given name and value.
 - `removeAttribute(String name)`: removes the attribute with the given name from the session.
 - `invalidate()`: invalidates the session and removes all the attributes.
 - `isNew()`: returns true if the session is new, i.e., the client has not yet received the session ID, or false otherwise.

Java Server Pages (JSP) in Servlets

- Java Server Pages (JSP) are a technology that allows web developers to create dynamic web pages using HTML, XML, or other document types.
- JSP are similar to servlets, but they have a simpler syntax and can be embedded in HTML pages.
- JSP are compiled into servlets by a JSP engine, which is part of a web server or a web container.
- JSP can use Java code, expressions, scriptlets, declarations, and directives to generate dynamic content.
- JSP can also use custom tags, which are reusable components that encapsulate Java code or other JSP elements.
- JSP support the Model-View-Controller (MVC) design pattern, which separates the business logic, the presentation, and the user interaction of a

web application.

- JSP can communicate with servlets, JavaBeans, databases, and other web resources using various mechanisms, such as request and response objects, session and application objects, cookies, URL rewriting, and filters.
- JSP have several advantages over servlets, such as:
 - Ease of development and maintenance, as JSP can separate the presentation from the logic.
 - Portability and compatibility, as JSP can run on any platform and web server that supports the Java Servlet API.
 - Performance and scalability, as JSP are compiled into servlets, which are fast and efficient.
 - Extensibility and reusability, as JSP can use custom tags and Java libraries to create modular and reusable web components.

Introduction to JSP in Servlets

- JSP stands for Java Server Pages. It is a server-side technology that is used to create dynamic web applications.
- JSP is an extension of Servlet technology, which is a Java-based API that allows processing and responding to HTTP requests.
- JSP uses HTML tags and JSP tags to create web pages. JSP tags are special tags that can insert Java code or expressions into HTML pages.
- JSP pages are translated into Servlets by a JSP translator or compiler. The Servlets are then executed by a web container or server, such as Tomcat or Jetty.
- JSP provides more functionality than Servlets, such as expression language, JSTL, custom tags, etc. Expression language is a simple syntax that allows accessing data and performing operations. JSTL is a set of standard tag libraries that provide common functionality, such as iteration, conditionals, formatting, etc. Custom tags are user-defined tags that can encapsulate reusable logic or components.
- JSP follows the MVC (Model-View-Controller) pattern, which separates the business logic, presentation logic, and data. The Model represents the data or the state of the application. The View represents the user interface or the output of the application. The Controller represents the logic or the behavior of the application. JSP is mainly used as the View component, while Servlets are used as the Controller component. The Model can be any Java object or data source, such as a database, a file, etc.

Java Server Pages Overview in Servlets

- JavaServer Pages (JSP) is a technology that allows developers to create dynamic web pages using Java and Java Servlets .
- JSP pages are stored as regular HTML files with a .jsp extension and can contain HTML, XML, JavaScript, CSS, and Java code snippets.
- JSP pages are compiled into Java servlets and run on the server-side by a

servlet container, such as Apache Tomcat or Jetty .

- JSP pages use a special syntax that embeds Java code within HTML tags, such as `<% ... %>` for scriptlets, `<%= ... %>` for expressions, and `<%@ ... %>` for directives .
- JSP pages can also use custom tags, which are reusable components that encapsulate Java logic and can be invoked by a simple tag name, such as .
- JSP pages can communicate with servlets and other web components using request and response objects, which provide access to HTTP headers, parameters, cookies, sessions, and other information .
- JSP pages can also use JavaBeans, which are reusable Java classes that follow a naming convention and can be accessed by JSP pages using and tags .
- JSP pages can be combined with servlets to implement the Model-View-Controller (MVC) pattern, where servlets handle the business logic and JSP pages handle the presentation logic.
- JSP pages have a life cycle that consists of the following phases: translation, compilation, initialization, execution, and destruction .
- JSP pages offer several advantages over servlets, such as ease of development, separation of concerns, and support for custom tags and JavaBeans .

A First Java Server Page Example in Servlets

- A Java Server Page (JSP) is a web page that contains Java code embedded in HTML or XML.
- A JSP is compiled into a servlet by a JSP compiler, which is a special type of servlet container.
- A servlet is a Java class that handles HTTP requests and generates dynamic web content.
- A JSP can use Java expressions, scriptlets, declarations, directives, and custom tags to create dynamic web content.
- A JSP can also access JavaBeans, servlets, and other Java components to perform business logic and data processing.
- A simple JSP example is shown below:

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
<head>
  <title>A First JSP Example</title>
</head>
<body>
  <h1>A First JSP Example</h1>
  <p>The current date and time is: <%= new java.util.Date() %></p>
```

```
<p>The value of pi is: <%= Math.PI %></p>
</body>
</html>
```

- The first line of the JSP is a directive that specifies the content type, the language, and other attributes of the page.
- The HTML tags create the structure and style of the web page.
- The `<%= %>` tags are expressions that evaluate to a value and insert it into the output stream.
- The new `java.util.Date()` expression creates a `Date` object and returns its string representation.
- The `Math.PI` expression returns the value of the constant `pi`.
- When this JSP is requested by a browser, the JSP compiler compiles it into a servlet class and executes it on the server.
- The servlet generates the HTML output and sends it back to the browser.
- The browser displays the web page with the dynamic content.

Implicit Objects in Servlets

- Implicit objects are Java objects that are created by the servlet container and are available to all the servlets within a web application.
- Implicit objects are also known as pre-defined variables or pre-defined objects.
- Implicit objects are used to access the information related to a particular request, response, session, application, or page.
- Implicit objects are automatically passed to the service method of the servlet by the container, so the servlet does not need to create or initialize them.
- There are nine implicit objects in servlets: `request`, `response`, `out`, `session`, `application`, `config`, `pageContext`, `page`, and `exception`.
- The `request` object represents the HTTP request sent by the client to the server. It contains the request parameters, headers, cookies, attributes, and other information.
- The `response` object represents the HTTP response sent by the server to the client. It contains the response status, headers, cookies, and output stream.
- The `out` object is a `PrintWriter` object that is used to send the output to the client. It is obtained from the `response` object using the `getWriter()` method.
- The `session` object represents the HTTP session associated with the request. It is used to store and retrieve the data related to a particular user across multiple requests.

- The application object represents the ServletContext object that is shared by all the servlets within a web application. It is used to store and retrieve the data related to the whole application across multiple sessions.
- The config object represents the ServletConfig object that is associated with a particular servlet. It is used to access the initialization parameters and the servlet context of the servlet.
- The pageContext object represents the PageContext object that is created for each JSP page. It is used to access the implicit objects, the page scope attributes, and the JSP expressions.
- The page object represents the current JSP page. It is equivalent to the this keyword in Java.
- The exception object represents the Throwable object that is thrown by the JSP page. It is only available in the error pages that have the isErrorPage attribute set to true.

Scripting in Servlets

- Scripting in servlets refers to the use of scripting elements such as declarations, scriptlets, and expressions to embed Java code within a JSP page.
- Scripting elements are delimited by special symbols: `<%` and `%>` for declarations and scriptlets, and `<%=` and `%>` for expressions.
- Declarations are used to declare variables and methods that can be used by other scripting elements in the same JSP page. They do not produce any output to the client.
- Scriptlets are used to write Java code that performs some logic or computation. They can access and modify the implicit objects and variables available in the JSP page. They can also produce output to the client by using the out object or by using expressions.
- Expressions are used to evaluate a Java expression and insert its value into the output stream. They are equivalent to using `out.print()` with the expression as the argument. Expressions cannot contain semicolons or assignment operators.
- Scripting elements can be mixed with HTML tags and other JSP elements to create dynamic web pages. However, they should be used sparingly as they can make the JSP page less readable and maintainable. It is recommended to use JSP actions, custom tags, and JavaBeans instead of scripting elements whenever possible.

Standard Actions in Servlets

- Standard actions are predefined tags that perform some common tasks in servlets, such as including other resources, forwarding requests, or handling errors.
- Standard actions start with `<jsp:` and end with `/>`. They have attributes that specify the parameters for the action.

- Some of the standard actions are:
 - **<jsp:include>**: This action includes the output of another resource, such as a servlet, JSP page, or HTML file, in the current JSP page. It has two attributes: **page**, which specifies the relative URL of the resource to be included, and **flush**, which indicates whether the output buffer should be flushed before including the resource. The default value of **flush** is **false**.
 - **<jsp:forward>**: This action forwards the current request to another resource, such as a servlet, JSP page, or HTML file. It has one attribute: **page**, which specifies the relative URL of the resource to be forwarded to. The action terminates the execution of the current JSP page and transfers the control to the forwarded resource.
 - **<jsp:param>**: This action is used to pass parameters to the **<jsp:include>** or **<jsp:forward>** actions. It has two attributes: **name**, which specifies the name of the parameter, and **value**, which specifies the value of the parameter. The action must be nested inside the **<jsp:include>** or **<jsp:forward>** tags.
 - **<jsp:plugin>**: This action generates the necessary HTML code to invoke a Java applet or a JavaBean component in the browser. It has several attributes, such as **type**, **code**, **codebase**, **width**, **height**, etc., that specify the details of the applet or the component. The action can also have nested **<jsp:params>** tags to pass parameters to the applet or the component.
 - **<jsp:useBean>**: This action creates or locates a JavaBean component and associates it with a scripting variable. It has four attributes: **id**, which specifies the name of the scripting variable, **class**, which specifies the fully qualified name of the JavaBean class, **scope**, which specifies the scope of the bean (page, request, session, or application), and **beanName**, which specifies the name of the bean as a String or a runtime expression. The action can also have a body that contains **<jsp:setProperty>** or **<jsp:getProperty>** tags to set or get the properties of the bean.
 - **<jsp:setProperty>**: This action sets the value of a property of a JavaBean component. It has three attributes: **name**, which specifies the name of the scripting variable that references the bean, **property**, which specifies the name of the property to be set, and **value**, which specifies the value of the property as a String or a runtime expression. The action can also use the **param** attribute to set the property value from a request parameter with the same name as the property.
 - **<jsp:getProperty>**: This action gets the value of a property of a JavaBean component and writes it to the output. It has two attributes: **name**, which specifies the name of the scripting variable that references the bean, and **property**, which specifies the name of the property to be retrieved.

Directives in Servlets

- Directives are instructions that control the structure and properties of a servlet.
- Directives are written in the servlet source code using the syntax `<%@ directive attribute="value" %>`.
- There are three types of directives in servlets: page, include and taglib.
- The page directive defines the attributes of the servlet, such as the content type, the language, the error page, the buffer size, etc.
- The include directive inserts the content of another file into the servlet at compile time. It is useful for reusing common code or HTML fragments.
- The taglib directive declares the use of a custom tag library in the servlet. It specifies the prefix and the URI of the tag library. Custom tags are user-defined tags that can perform complex tasks or encapsulate reusable functionality.

Custom Tag Libraries in Servlets

- Custom tag libraries are a way of creating reusable components in JSP pages.
- Custom tag libraries can encapsulate complex logic, presentation, or functionality that can be used across multiple JSP pages or applications.
- Custom tag libraries are defined in a tag library descriptor (TLD) file, which specifies the name, attributes, and implementation class of each custom tag.
- Custom tag libraries can be accessed from a JSP page by using a taglib directive, which declares the URI and prefix of the tag library.
- Custom tag libraries can be implemented in two ways: as simple tags or as classic tags.
- Simple tags are easier to write and use, as they extend the `javax.servlet.jsp.tagext.SimpleTagSupport` class and override the `doTag()` method.
- Classic tags are more flexible and powerful, as they implement the `javax.servlet.jsp.tagext.Tag` interface and define the lifecycle methods of the tag.
- Custom tag libraries can also use tag files, which are JSP fragments that define the body and attributes of a custom tag. Tag files are stored in the `WEB-INF/tags` directory or a subdirectory of it.